TEST PLAN

GENERAL COMMENTS

1. Firstly, we worked separately and then we discussed our solutions and troubles. That process happened multiple times and each time we reviewed our issues. The final scripts are the result of our combined effort.

2. There were some processes that needed trial and error, and we had to drop or delete tables multiple times, constraints, triggers and sequences: The number of integers or the characters of the attributes were initially too small, so we made adjustments accordingly. Also, we needed to update some values for the queries to return "desired" results.

3. We created a file with the lines of code that we run at the beginning of each session. That txt file has the login procedure, the formatting of the date and the lines that we use for the queries to return compactly the results, thus optimizing readability. We also included the queries and the code for getting the metadata about our schema (seeing triggers, constraints, procedures, tables, sequences, and the number of insertions on our tables).

4. Our AI tools were Copilot and ChatGPT. We used them restrictively and as a guide, not a solution. We list all the major prompts of AI's help. We used AI's help for some minor specific issues, but we didn't include the prompts for space saving purposes

SUCCESSFUL SCHEMA CREATION AND DELETION

Table A. In this table, we list all the process of building and dropping our schema. Apart from the information listed we refer to Figures from Appendix A. It is important to note that this table contains only clean successful code.

ID	TEST	ELEMENT TESTED	RESULT	FIG

ACR01	`CREATE SEQUENCE <SEQ_NAME>`
	`  INCREMENT BY 1`
	`  START WITH ###`
	`  MAXVALUE ###;`
		sequences	4 sequences created	1

ACR02	`SELECT * FROM USER_SEQUENCES;`	sequences	Checking the creation of sequences	2
ACR03	`CREATE TABLE <TAB_NAME>(` `  COL_NAME DATA_TYPE (NOT NULL) (DEFAULT));`	tables	5 tables created	3
ACR04	`SELECT * FROM TAB` `WHERE TNAME NOT LIKE 'BIN%';`	tables	Checking the creation of tables	4
ACR05	`DESC FOOTBALLER;`	Footballer table	Description of footballer table's attributes	5
ACS01	`ALTER TABLE <TAB> ADD (` `CONSTRAINT …);`	constraints	Table alterations, creation of 15 constraints	6

ACS02	`SELECT CONSTRAINT_NAME FROM USER_CONSTRAINTS`
	`WHERE CONSTRAINT_NAME NOT LIKE 'BIN%' AND`
	`CONSTRAINT_NAME NOT LIKE 'SYS%';`
		constraints	Checking the 15 constraints	7

ACS03	`CREATE TRIGGER <TRG_NAME>`
	`<BEFORE-AFTER> <INSERT OR UPDATE> ON <TAB>`
	`FOR EACH ROW`
	`BEGIN`
	`  <…>`
	`END;`
	`/`
		triggers	5 triggers created	8

ACS04	CREATE PROCEDURE <PRC_NAME>(
	  <ARGUMENTS> <DATA_TYPE>) AS
	  <VARIABLE_NAMES> <DATA_TYPES>;
	BEGIN
	  <…>
	END;
	/
		procedure	1 procedure created, 1 procedure executed and 4 triggers created	9

ACS05	SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE 'PRC%';	procedure	Checking the procedure created	10
ACS06	SELECT TRIGGER_NAME FROM USER_TRIGGERS WHERE TRIGGER_NAME LIKE 'TRG%';	triggers	Checking the 9, in total, triggers created	11
AIN01	INSERT INTO STADIUM <COL_NAMES> VALUES <…>; INSERT INTO STADIUM VALUES <…>;	Stadium table	Population of stadium table with 8 rows	12
AIN02	INSERT INTO TEAM <COL_NAMES> VALUES <…>; INSERT INTO TEAM VALUES <…>;	Team table	Population of team table with 14 rows	13
AIN03	INSERT INTO FOOTBALLER <COL_NAMES> VALUES <…>; INSERT INTO STADIUM VALUES <…>;	Footballer table	Population of footballer table with 40 rows	14
AIN04	INSERT INTO SPONSOR <COL_NAMES> VALUES <…>; INSERT INTO SPONSOR VALUES <…>;	Sponsor table	Population of sponsor table with 9 rows	15
AIN05	INSERT INTO SPONSORSHIP <COL_NAMES> VALUES <…>; INSERT INTO SPONSORSHIP VALUES <…>;	Sponsorship table	Population of sponsorship table with 20 rows	16
AIN06	SELECT COUNT(*) FROM <TAB>;	All tables	The number of rows of each table	17

AFM01	CREATE PROCEDURE <PRC_NAME>(
	  <ARGUMENTS> <DATA_TYPE>) AS
	  <VARIABLE_NAMES> <DATA_TYPES>;
	BEGIN
	  <…>
	END;
	/
		procedure	2 procedures creating for formatting	18

AFM02	EXEC PRC_START;	procedure	We get the lines that we need for formatting at the start of the session	19
AFM03	SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE ‘PRC%’;	procedures	We get our procedures	20
AQR01	SELECT * FROM TEAM;	Query A	All the records from table TEAM	21
AQR02	SELECT ID, NAME, CITY_FOUNDED, GOALS_SCORED FROM TEAM ORDER BY CITY_FOUNDED DESC, NAME DESC;	Query B	A projection of columns ID, NAME, CITY_FOUNDED in reverse alphabetical order	22
AQR03	SELECT FIRST_NAME, SURNAME, COUNTRY FROM FOOTBALLER WHERE COUNTRY='GREECE';	Query C1	Players from Greece	23
AQR04	SELECT FIRST_NAME, SURNAME FROM FOOTBALLER WHERE SURNAME LIKE 'J%SON';	Query C2	Players that their surname begins with J and ends with SON	24
AQR05	SELECT FIRST_NAME, SURNAME, COUNTRY FROM FOOTBALLER WHERE COUNTRY<>'ENGLAND';	Query C3	Players who are not from England	25
AQR06	SELECT <COLS>, TO_CHAR(DOB, 'MONTH') FROM FOOTBALLER WHERE TO_NUMBER(TO_CHAR(DOB, 'MM')) BETWEEN 4 AND 9;	Query C4	Players who are born from April to September	26
AQR07	SELECT SURNAME, COUNTRY, POSITION FROM FOOTBALLER WHERE COUNTRY IN ('BRAZIL', 'ENGLAND') AND POSITION IN ('GK', 'MID') AND SURNAME LIKE 'L%' AND SURNAME NOT LIKE '%i%' ORDER BY COUNTRY, POSITION;	Query C5	Players thar either from England or Brazil and their position are either goal keeper or mid and their surname starts with L and doesn’t contain i	27
AQR08	SELECT T.NAME, T.GOALS_SCORED, F.SURNAME, F.GOALS FROM TEAM T, FOOTBALLER F WHERE T.ID=F.TEAM_ID AND F.COUNTRY IN ('ENGLAND','BRAZIL');	Query D1	Join of tables TEAM and FOOTBALLER	28
AQR09	SELECT S.NAME, S.CITY, T.NAME, T.CITY_FOUNDED, F.SURNAME, F.COUNTRY FROM FOOTBALLER F, TEAM T, STADIUM S WHERE T.ID=F.TEAM_ID AND S.ID=T.STADIUM_ID AND F.CONTRACT>70000 AND COUNTRY NOT IN ('BRAZIL', 'GREECE', 'ENGLAND'); SELECT TM.NAME, SP.NAME, SS.FEE	Query D2	Join of tables FOOTBALLER, TEAM AND STADIUM Join of tables SPONSORSHIP, TEAM AND SPONSOR	29

	FROM SPONSORSHIP SS JOIN TEAM TM ON SS.TEAM_ID = TM.ID JOIN SPONSOR SP ON SS.SPONSOR_ID = SP.ID ORDER BY SS.FEE DESC;			
AQR10	SELECT SP.NAME, TM.NAME, ST.NAME, COUNT(FB.ID), SS.FEE FROM STADIUM ST, TEAM TM, FOOTBALLER FB, SPONSOR SP, SPONSORSHIP SS WHERE SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND TM.ID=FB.TEAM_ID AND ST.ID=TM.STADIUM_ID GROUP BY SP.NAME, TM.NAME, ST.NAME, SS.FEE;	Query D3	Join of all the 5 tables	30
AQR11	SELECT FIRST_NAME, SURNAME, COUNTRY FROM FOOTBALLER WHERE SURNAME LIKE 'K%S';	Query E1	Players that their surname starts with K and ends with S	31
AQR12	SELECT T.NAME, COUNT(*) PLAYERS FROM FOOTBALLER F, TEAM T WHERE T.ID=F.TEAM_ID GROUP BY T.NAME; SELECT T.ID, T.NAME, COUNT(F.SURNAME) NUMBER_OF_PLAYERS FROM FOOTBALLER F JOIN TEAM T ON F.TEAM_ID = T.ID GROUP BY T.NAME, T.ID;	Query E2	Total number of players per team (2 ways)	32
AQR13	SELECT SURNAME, FIRST_NAME, CONTRACT FROM FOOTBALLER WHERE CONTRACT=(SELECT MAX(CONTRACT) FROM FOOTBALLER);	Query E3	Highest paid player	33
AQR14	SELECT COUNT(*) COUNT_RS_LESS_2 FROM STADIUM WHERE REVIEW_SCORE<2;	Query E4	Number of stadiums not hosting a team	34
AQR15	SELECT F.SURNAME, F.FIRST_NAME, F.ID, T.NAME FROM FOOTBALLER F, TEAM T WHERE T.ID=F.TEAM_ID ORDER BY SURNAME ASC, FIRST_NAME ASC;	Query E5	Players' SURNAME, FIRST_NAME and ID and their team in ascending order	35
AQR16	SELECT F.SURNAME, F.FIRST_NAME, T.NAME, F.GOALS FROM FOOTBALLER F, TEAM T WHERE T.ID=F.TEAM_ID AND F.GOALS=(SELECT MAX(GOALS) FROM FOOTBALLER);	Query E6	Player with most goals	36
AQR17	SELECT CITY_FOUNDED, COUNT(*) TEAMS_PER_CITY FROM TEAM GROUP BY CITY_FOUNDED ORDER BY COUNT(*) DESC, CITY_FOUNDED ASC;	Query E7	Total number of teams per city	37
AQR18	SELECT NAME, GOALS_SCORED-GOALS_CONCEDED DIFF FROM TEAM WHERE (GOALS_SCORED-GOALS_CONCEDED) = (SELECT MIN(GOALS_SCORED -GOALS_CONCEDED) FROM TEAM);	Query E8	Team with the worst goal difference	38
AQR19	SELECT COUNT(*) COUNT_YEARBUILT_1980_1999 FROM STADIUM WHERE YEAR_BUILT BETWEEN 1980 AND 1999; SELECT COUNT(NAME) AS COUNT_OF_STADIUMS_1980_1999 FROM STADIUM WHERE YEAR_BUILT >= 1980 AND YEAR_BUILT <= 1999;	Query E9	Number of stadiums built between 1980 and 1999 (2 ways)	39
AQR20	SELECT FIRST_NAME, SURNAME, COUNTRY, DOB FROM FOOTBALLER WHERE TO_CHAR(DOB, 'MON')='JUL' ORDER BY TO_CHAR(DOB,'DD') ASC; SELECT FIRST_NAME, SURNAME, DOB FROM FOOTBALLER WHERE TO_NUMBER(TO_CHAR(DOB, 'MM')) = 7;	Query E10	Players that were born in July (2 ways)	40
AQR21	SELECT COUNT(*) NUMBER_OF_STADIUMS_NOT_HOSTING_TEAMS FROM STADIUM S WHERE S.ID NOT IN (SELECT DISTINCT T.STADIUM_ID FROM TEAM T);	Query E11	Number of stadiums not hosting a team	41
AQR22	SELECT NAME FROM TEAM WHERE NAME LIKE '%SAINT%';	Query E12	Teams that their name contains 'saint'	42
AQR23	SELECT FIRST_NAME, SURNAME, CONTRACT, GOALS FROM FOOTBALLER	Query E13	Player(s) from Brazil who has the highest contract or scored	43

	WHERE COUNTRY='BRAZIL' AND (CONTRACT=(SELECT MAX(CONTRACT) FROM FOOTBALLER WHERE COUNTRY='BRAZIL') OR GOALS=(SELECT MAX(GOALS) FROM FOOTBALLER WHERE COUNTRY='BRAZIL')); -- VERIFICATION SELECT FIRST_NAME, SURNAME, CONTRACT, GOALS FROM FOOTBALLER WHERE COUNTRY='BRAZIL';		the most goals (with verification query)	
AQR24	SELECT F.SURNAME, F.FIRST_NAME, F.POSITION, F.CONTRACT FROM FOOTBALLER F, (SELECT POSITION POS, MAX(CONTRACT) MAXCON FROM FOOTBALLER GROUP BY POSITION) M WHERE F.POSITION=POS AND F.CONTRACT=MAXCON; SELECT F.FIRST_NAME, F.SURNAME, F.POSITION, F.CONTRACT FROM FOOTBALLER F WHERE F.CONTRACT = (SELECT MAX(F2.CONTRACT) FROM FOOTBALLER F2 WHERE F2.POSITION = F.POSITION) ORDER BY F.POSITION;	Query E14	Players with highest contract per position (2 ways)	44
AQR25	SELECT ID, SUBSTR(COUNTRY,1,2) || SUBSTR(FIRST_NAME,1,4) || SUBSTR(SURNAME,1,4) USERNAME FROM FOOTBALLER;	Query E15	Players' ID and username (2 first letters of country, 4 first letters of their first name and 4 first letters of their surname	45
AQR26	SELECT SUM(CONTRACT) TOTAL_CONTRACT_AMOUNT FROM FOOTBALLER;	Query E16	Players' total contract	46
AQR27	SELECT SP.NAME, SUM(SS.FEE) FROM SPONSOR SP, SPONSORSHIP SS WHERE SP.ID=SS.SPONSOR_ID GROUP BY SP.NAME HAVING SUM(SS.FEE)=(SELECT MAX(SUMFEE) FROM (SELECT SUM(SS.FEE) SUMFEE FROM SPONSORSHIP SS GROUP BY SS.SPONSOR_ID) M);	Query E17	Sponsor spending most money advertising	47
AQR28	SELECT SP.TYPE, SUM(SS.FEE) FROM SPONSOR SP, SPONSORSHIP SS WHERE SP.ID=SS.SPONSOR_ID GROUP BY SP.TYPE HAVING SUM(SS.FEE)=(SELECT MAX(XSUMFEE) FROM (SELECT SUM(SS.FEE) XSUMFEE FROM SPONSOR SP, SPONSORSHIP SS WHERE SP.ID=SS.SPONSOR_ID GROUP BY SP.TYPE) X);	Query E18	Type of business that spends most on advertising	48
AQR29	SELECT FB.FIRST_NAME, FB.SURNAME, TM.NAME, FB.CONTRACT, SP.NAME FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND SP.ID IN (SELECT SP.ID FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID GROUP BY SP.ID HAVING AVG(FB.CONTRACT)=(SELECT MAX(AVGCON) FROM (SELECT AVG(FB.CONTRACT) AVGCON FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID GROUP BY SP.ID))); --VERIFICATION SELECT SP.NAME, AVG(CONTRACT) FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID GROUP BY SP.NAME ORDER BY AVG(CONTRACT) DESC;	Query E19	Players with highest average contract per sponsor (with verification query)	49

AQR30	SELECT TM.NAME, SUM(NVL(SS.FEE,0)) FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS' GROUP BY TM.NAME HAVING SUM(SS.FEE)=(SELECT MAX(XSUMFEE) FROM (SELECT SUM(NVL(SS.FEE,0)) XSUMFEE FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS' GROUP BY TM.NAME) X); -- VERIFICATION SELECT TM.NAME, SUM(NVL(SS.FEE,0)) FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS' GROUP BY TM.NAME ORDER BY SUM(NVL(SS.FEE,0)) DESC;	Query E20	Team that gets most money from electronics sponsors (with verification query)	50
ADR01	DROP PROCEDURE <PRC>;	Procedures	All procedures are dropped	51
ADR02	DROP TABLE <TAB> CASCADE CONSTRAINTS;	Tables, constraints, triggers	All the tables, constraints and triggers are dropped	52
ADR03	DROP SEQUENCE <SEQ>;	Sequences	All the sequences are dropped	53
ADR04	SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE 'PRC%'; SELECT TRIGGER_NAME FROM USER_TRIGGERS WHERE TRIGGER_NAME LIKE 'TRG%'; SELECT CONSTRAINT_NAME FROM USER_CONSTRAINTS WHERE CONSTRAINT_NAME NOT LIKE 'BIN%' AND CONSTRAINT_NAME NOT LIKE 'SYS%'; SELECT TNAME FROM TAB WHERE TNAME NOT LIKE 'BIN%'; SELECT SEQUENCE_NAME FROM USER_SEQUENCES;	Whole schema	Checking that indeed everything is dropped from the schema	54

ISSUES AND AI'S HELP

Table B. We organize and state our issues and AI's help on this table. We refer to Appendix B with the Figures showing either our issue or the AI's help. It is important to note that in this table we do not list successful code and points, but only the issues and how we tackled them.

ID	INITIAL TEST	ELEMENT TESTED	ISSUES	UNDERSTANDING	FIG
BCR01	NOT NULL	Foreign keys	Clarification	Due to ERD, every team must be hosted in a stadium, every player must be in a team, and every sponsorship must be sponsored by a specific team and sponsor. Only stadiums can be able not to host a team. For these reasons, every foreign key must be not null. If, for example, a team could be able not to be hosted in a stadium, then this foreign key could be null.	55
BCS01	ALTER TABLE <TAB> ADD CONSTRAINT <CONSTRAINT>;	Constraints	Optional help on writing more compact code	We can ADD CONSTRAINTS per table in a more compact way with the following syntax: ALTER TABLE <TAB> ADD(CONSTRAINT <CONSTRAINT>, ..., CONSTRAINT <CONSTRAINT>);	56

BCS02	ALTER TABLE <TAB> ADD CONSTRAINT CHK_UPP_### CHECK (COL=UPPER(COL));	Case of certain attributes	Case restriction of insertions with check uppercase constraints	We can construct a BEFORE INSERT-UPDATE TRIGGER to ensure that the new value is inserted in uppercase	57
BCS03	--	Triggers	We wanted to construct triggers and since they were out of the curriculum of the course, we explored their syntax with AI's help	We found out the structure of triggers and the main keywords and clauses so we can be able to use them. We later proceed to ask some minor questions regarding the structure of a trigger (when to put a semi colon, how :NEW and :OLD are used syntactically, etc.)	58
BCS04	Example of Temporal Check Constraint: ALTER TABLE STADIUM ADD CONSTRAINT CHK_STADIUM_YEARBUILT CHECK (TO_CHAR(YEAR_BUILT) <= TO_CHAR(SYSDATE, 'YYYY'));	Constraints and Triggers	We tried to construct constraints that ensure valid values for temporal attributes.	We found out that it is not possible to use a variable value to construct a constraint (SYSDATE). That's why we ended up constructing triggers for our temporal attributes in order to ensure valid values.	59
BCS05	ALTER TABLE FOOTABLLER ADD CONSTRAINT CHK_FOOTBALLER_POSITION CHECK (POSITION IN ('GK','DEF','MID','FOR'));	Constraints	Clarification	We do not need another constraint or trigger to ensure the upper case for the POSITION, given this constraint	60

BCS06	CREATE TRIGGER TRG_TEAM_GOALS
	AFTER INSERT OR UPDATE ON FOOTBALLER
	FOR EACH ROW
	DECLARE
	  GOALS_SUM NUMBER;
	BEGIN
	  SELECT NVL(SUM(F.GOALS),0)
	  INTO GOALS_SUM
	  FROM FOOTBALLER F
	  WHERE F.TEAM_ID = :NEW.TEAM_ID;
	
	  UPDATE TEAM T
	  SET T.GOALS_SCORED = GOALS_SUM
	  WHERE T.ID = :NEW.TEAM_ID AND GOALS_SUM > T.GOALS_SCORED;
	END;
	/
		Trigger	See Figure 61.	We cannot query a table that is currently modified by an insertion or update in a trigger. That is why our trigger didn't work. We ended up constructing a simpler trigger that bypasses this issue.	62

BCS07	--	Procedures	We wanted to construct a procedure that builds the upper-case triggers for three reasons: a. to expand our knowledge of procedures, b. not to construct similar repetitive triggers and c. to enrich our assignment	We learnt the basic structure of a procedure. We learnt about the specific clause that we will use. We did some extra questions to AI regarding the DECLARE, EXECUTE IMMEDIATE and some other minor syntactical questions (AS-IS, if DECLARE is needed, what is a list object of Oracle and how to make a for loop in Oracle).	63
BIN01	To populate tables by hand	Tables' Population	We didn't want to populate the tables by hand since the process was too time-consuming and tedious. Also, we knew beforehand that we would need to update the tables several	We gave the description of our schema to AI along with the queries so it can generate values for our tables' population, such that the queries would not return <no rows>. We used AI's help only to get the values, but not to write the code. We didn't provide a screenshot since it was too lengthy for space saving purposes.	--

			times to have results in our queries.		
BIN02	INSERT INTO <TAB> VALUES <…>;	Insert	We wanted to know if there is a more compact way of inserting values, like we did for the constraints	No there is no more compact way to insert values in this version of Oracle	64
BFM01	SET WRAP OFF	Formatting	When we tried to query multiple columns then, either the columns were overlapping or they were off screen. with a lot of wasted space.	We learnt how to format each session, so the columns will be more compact. We also learnt how to format the size of the columns.	65

BFM02	CREATE OR REPLACE PROCEDURE PRC_COLS_FORMAT(TAB_NAME VARCHAR2) AS
	QUERY_MAX_COLS_LEN VARCHAR2(200);
	MAX_COLS_LEN NUMBER;
	FORMATTING VARCHAR2(20);
	COL_FORMAT VARCHAR2(40);
	BEGIN
	FOR COLS IN (SELECT COLUMN_NAME AS NAME, DATA_TYPE AS TYPE FROM USER_TAB_COLUMNS WHERE TABLE_NAME = UPPER(TAB_NAME) ORDER BY COLUMN_ID) LOOP
	QUERY_MAX_COLS_LEN := 'SELECT MAX(LENGTH(' || cols.name || ')) FROM ' || TAB_NAME;
	EXECUTE IMMEDIATE QUERY_MAX_COLS_LEN INTO MAX_COLS_LEN;
	FORMATTING:=CASE
	WHEN COLS.TYPE:='NUMBER' THEN LPAD('9', MAX_COLS_LEN, '9')
	ELSE 'A'||TO_CHAR(MAX_COLS_LEN)
	END;
	COL_FORMAT:='COLUMN '||COLS.NAME||' FORMAT '||FORMATTING
	EXECUTE IMMEDIATE COL_FORMAT;
	END LOOP;
	END;
	/
		Procedure	We built two procedures that generate the formatting lines and run them. However, formatting lines don't run like queries and when stored in a text variable cannot be executed with EXECUTE IMMEDIATE.	We understood that we cannot execute formatting lines like queries. Also, we learnt how to print-output text with DBMS_OUTPUT.PUT_LINE. We decided to have a procedure that generates the formatting text of the columns and a procedure that wraps the first one and generates the formatting text of the session. Calling the latter will automatically generate all the formatting text. We either can store the text in a script and copy, paste and run it from the script, or we can execute the procedure at the start of the session, copy, paste and run it from the command line.	66

BQR01	WHERE <join equalities> JOIN <tab> ON <join equality>	JOIN ON	We had some issues with the modern way of joining tables with the JOIN ON syntax, especially in the case that we needed to join multiple tables with potentially more than one dependency per table.	We learnt the way to join tables with the JOIN ON clause. For example: SELECT <COLS> FROM stadium st JOIN team tm ON tm.STADIUM_ID=st.ID JOIN sponsorship ss ON ss.TEAM_ID=tm.ID JOIN sponsor sp ON sp.ID=ss.SPONSOR_ID	67
BQR02	We wanted to get max-min values, and we knew how to do that with subqueries. But we wanted to know if we can sort and take the first row with LIMIT.	LIMIT	Clarification on whether LIMIT is available in oracle	In oracle there is FETCH FIRST n ROWS ONLY and ROWNUM	68

BQR03	We know how to syntactically have a subquery in a WHERE and HAVING clauses and we know what it means to put a subquery in these clauses.	WHERE – HAVING Query E17 Query E18 Query E19 Query E20	Clarification on when to put a subquery in the clauses (similarities and differences)	If the filtering condition includes aggregation based on a GROUP BY statement, then this subquery must be put on a HAVING clause. If not and the filtering condition is irrelevant to the aggregation (before GROUP BY) then the subquery is put in a WHERE clause.	69
BQR04	We know how to syntactically have a subquery in a FROM and WHERE clauses and we know what it means to put a subquery in these clauses	FROM – WHERE Query E14 Query E17 Query E18 Query E19 Query E20	Clarification on when to put a subquery in the clauses (similarities and differences)	If we need to join a subquery pre aggregated table with another table, then it is best to put a subquery like this in a FROM clause. Also, if we need to use multiple columns of a subquery, it is recommended to put it in a FROM clause. Otherwise, when we have filtering conditions (especially on a certain value or comparing columns) a WHERE clause is preferred	70
BQR05	SELECT T. NAME, T. CITY_FOUNDED, AVG(F.CONTRACT) FROM TEAM T, FOOTBALLER F WHERE T.ID=F.TEAM_ID GROUP BY T.NAME ORDER BY AVG(F.CONTRACT);	GROUP BY	We had an issue with the GROUP BY query.	In the GROUP BY query, we must group all the non-aggregated columns stating them in the GROUP BY clause	71
BQR06	ORDER BY <col> ASC	Query E5	Clarification	ASC is optional since it is the default sorting mode, but we stated it explicitly for good measure.	72

BQR07	SELECT SP.NAME, SUM(SS.FEE)
	FROM SPONSOR SP, SPONSORSHIP SS
	WHERE SP.ID=SS.SPONSOR_ID
	GROUP BY SP.NAME
	HAVING SUM(SS.FEE)=MAX(
	  SELECT SUM(SS.FEE)
	  FROM SPONSORSHIP SS GROUP BY SS.SPONSOR_ID);
	
	SELECT SP.NAME, SUM(SS.FEE)
	FROM SPONSOR SP, SPONSORSHIP SS
	WHERE SP.ID=SS.SPONSOR_ID AND SUM(SS.FEE)=(
	  SELECT MAX(M.SUMFEE)
	  FROM (SELECT SUM(SS.FEE) SUMFEE FROM SPONSORSHIP SS GROUP BY SS.SPONSOR_ID) M
	);
		Query E17	We were having trouble finding a way to put correctly a subquery in a FROM, WHERE, HAVING clause	With the help of AI (BQR04) and with some trial and error we got the correct answer. We opt for a sum aggregation query with a HAVING filter containing a subquery there for the max needed value. In these harder queries, we truly understood when to put subqueries in a FROM, WHERE, HAVING clauses.	--

BQR08	SELECT XTYPE, MAX(XFEE)
	FROM (
	  SELECT SP.TYPE XTYPE, SUM(SS.FEE) XFEE
	  FROM SPONSOR SP, SPONSORSHIP SS
	  WHERE SP.ID=SS.SPONSOR_ID
	  GROUP BY SP.TYPE
	) X;
		Query E18	The same issue here with BQR07	With the help of AI (BQR03 and BQR04) and with some trial and error we managed to construct an aggregation query with a HAVING clause containing a subquery that fetches the desired aggregate value	--

BQR09	SELECT FB.FIRST_NAME, FB.SURNAME, TM.NAME, FB.CONTRACT, AVG(FB.CONTRACT)
	FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS, (
	  SELECT XID
	  FROM (
	  SELECT AVG(FB.CONTRACT) XAVG_CONTRACT, SP.ID XID
	  FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS
	  WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID
		Query E19: Players with the highest average contract per sponsor	The query was more complex. We had the same issues with the queries above. We also had some trouble grasping what we should do conceptually. We used AI to make specific questions about E19. We never asked for a ready code	After much trial and error, we made this conceptual workflow:
				1. The query stated that we need to fetch players' info in relation to sponsors. So, we needed to make a join table with footballer, team, sponsor and sponsorship (for the connection player-sponsor to take place).
				2. We aggregated the join table grouped by sponsor selecting the sponsor aiming for the average contract to be max with a subquery in a having clause.
					--

	GROUP BY SP.ID) X WHERE XAVG_CONTRACT=MAX(AVG_CONTRACT)) M WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND SP.ID=XID; SELECT FB.FIRST_NAME, FB.SURNAME, TM.NAME, FB.CONTRACT, SP.NAME FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS, (SELECT MAX(AVGCON), XID FROM (SELECT AVG(FB.CONTRACT) AVGCON, SP.ID XID FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID GROUP BY SP.ID) X GROUP BY XID) M WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND SP.ID=XID; SELECT FB.FIRST_NAME, FB.SURNAME, AVG(FB.CONTRACT) AVG_CONTRACT_PER_SPONSOR -- MAYBE MISSES A JOIN; FROM FOOTBALLER FB JOIN TEAM TM ON FB.TEAM_ID=TM.ID JOIN SPONSORSHIP SS ON SS.TEAM_ID=TM.ID JOIN SPONSOR SP ON SP.ID=SS.SPONSOR_ID GROUP BY FB.ID, FB.FIRST_NAME, FB.SURNAME ORDER BY AVG(FB.CONTRACT) DESC;		answer as we really wanted to get it by ourselves in the end.	So, the steps from inside-out must be: 1. Get the average contract values of the big join table grouped by sponsor 2. Get the max out of the previous subquery 3. Get the sponsor from the big join table grouped by the sponsor filtering with the max value from the previous subquery using a HAVING clause. 4. From the big join table grouped by sponsor, filter only these players associated with this sponsor in a WHERE clause	
BQR10	SELECT TM.NAME, MAX(XFEES) FROM TEAM TM, (SELECT SS.TEAM_ID XID, SUM(SS.FEE) XFEES FROM SPONSORSHIP SS, SPONSOR SP WHERE SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS' GROUP BY SS.TEAM_ID) X WHERE TM.ID=XID;	Query E20	We again had some issues with using correctly the subqueries in FROM-WHERE-HAVING clause	With the help of AI (BQR03 and BQR04) and some trial and error we worked like the previous query E19. It was a lot easier to follow a similar way of thinking since we understood the issues we had on E19. We created a big join table with team, sponsor and sponsorship and then: 1. Innermost subquery: we got the total amounts per team by the electronics sponsors 2. Inner subquery: we got the max value from the innermost subquery 3. Outer query: we found the team with the max amount of money with a HAVING clause using the max value from the inner subquery.	--

APPENDIX A

```
SQL> CREATE SEQUENCE SEQ_FOOTBALLER_ID
  2      INCREMENT BY 1
  3      START WITH 900
  4      MAXVALUE 999;

Sequence created.

SQL>
SQL> CREATE SEQUENCE SEQ_STADIUM_ID
  2      INCREMENT BY 1
  3      START WITH 200
  4      MAXVALUE 299;

Sequence created.

SQL>
SQL> CREATE SEQUENCE SEQ_TEAM_ID
  2      INCREMENT BY 1
  3      START WITH 100
  4      MAXVALUE 199;

Sequence created.

SQL>
SQL> CREATE SEQUENCE SEQ_SPONSOR_ID
  2      INCREMENT BY 1
  3      START WITH 400
  4      MAXVALUE 499;

Sequence created.
```

Figure 1. Creation of sequences

```
SQL> SELECT SEQUENCE_NAME FROM USER_SEQUENCES;

SEQUENCE_NAME
------------------------------------------------------------------------------------------------------------------------
SEQ_FOOTBALLER_ID
SEQ_SPONSOR_ID
SEQ_STADIUM_ID
SEQ_TEAM_ID
```

Figure 2. Checking the sequences.

```
SQL> CREATE TABLE STADIUM(
  2      ID NUMBER(3) DEFAULT SEQ_STADIUM_ID.NEXTVAL,
  3      NAME VARCHAR2(30) NOT NULL,
  4      CITY VARCHAR2(30) NOT NULL,
  5      CAPACITY NUMBER(6) DEFAULT 10000,
  6      YEAR_BUILT NUMBER(4) NOT NULL,
  7      REVIEW_SCORE NUMBER(1) DEFAULT 1);

Table created.

SQL>
SQL> CREATE TABLE TEAM(
  2      ID NUMBER(3) DEFAULT SEQ_TEAM_ID.NEXTVAL,
  3      NAME VARCHAR2(30) NOT NULL,
  4      CITY_FOUNDED VARCHAR2(30) NOT NULL,
  5      YEAR_FOUNDED NUMBER(4),
  6      GOALS_SCORED NUMBER(4),
  7      GOALS_CONCEDED NUMBER(4),
  8      STADIUM_ID NUMBER(3) NOT NULL);

Table created.

SQL>
SQL> CREATE TABLE FOOTBALLER(
  2      ID NUMBER(3) DEFAULT SEQ_FOOTBALLER_ID.NEXTVAL,
  3      FIRST_NAME VARCHAR2(20) NOT NULL,
  4      SURNAME VARCHAR2(20) NOT NULL,
  5      POSITION VARCHAR2(3) DEFAULT 'DEF',
  6      GAMES_PLAYED NUMBER(4),
  7      GOALS NUMBER(3) NOT NULL,
  8      CONTRACT NUMBER(8),
  9      EMAIL VARCHAR2(40) NOT NULL,
 10      COUNTRY VARCHAR2(30) NOT NULL,
 11      DOB DATE,
 12      TEAM_ID NUMBER(3) NOT NULL);

Table created.

SQL>
SQL> CREATE TABLE SPONSOR(
  2      ID NUMBER(3) DEFAULT SEQ_SPONSOR_ID.NEXTVAL,
  3      NAME VARCHAR2(30) NOT NULL,
  4      TYPE VARCHAR2(30) NOT NULL,
  5      CITY_HQ VARCHAR2(30) NOT NULL);

Table created.

SQL>
SQL> CREATE TABLE SPONSORSHIP(
  2      TEAM_ID NUMBER(3),
  3      SPONSOR_ID NUMBER(3),
  4      DEPOSIT_DATE DATE,
  5      FEE NUMBER(10));

Table created.
```

Figure 3. Creation of tables.

```
SQL> select * from tab
  2  where tname not like 'BIN%';

TNAME                                                
```

```
SQL> CREATE TRIGGER TRG_SPONSORSHIP_DEPOSITDATE
  2  BEFORE INSERT OR UPDATE ON SPONSORSHIP
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.DEPOSIT_DATE > SYSDATE THEN
  6          :NEW.DEPOSIT_DATE := SYSDATE;
  7      END IF;
  8  END;
  9  /

Trigger created.

SQL>
SQL> CREATE TRIGGER TRG_STADIUM_YEARBUILT
  2  BEFORE INSERT OR UPDATE ON STADIUM
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.YEAR_BUILT > TO_NUMBER(TO_CHAR(SYSDATE,'YYYY')) THEN
  6          :NEW.YEAR_BUILT := TO_NUMBER(TO_CHAR(SYSDATE,'YYYY'));
  7      END IF;
  8  END;
  9  /

Trigger created.

SQL>
SQL> CREATE TRIGGER TRG_TEAM_YEARFOUNDED
  2  BEFORE INSERT OR UPDATE ON TEAM
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.YEAR_FOUNDED > TO_NUMBER(TO_CHAR(SYSDATE,'YYYY')) THEN
  6          :NEW.YEAR_FOUNDED := TO_NUMBER(TO_CHAR(SYSDATE,'YYYY'));
  7      END IF;
  8  END;
  9  /

Trigger created.

SQL>
SQL> CREATE TRIGGER TRG_FOOTBALLER_DOB
  2  BEFORE INSERT OR UPDATE ON FOOTBALLER
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.DOB < ADD_MONTHS(SYSDATE, -12*100) OR :NEW.DOB > ADD_MONTHS(SYSDATE, -12*15) THEN
  6          RAISE_APPLICATION_ERROR(-20003, 'DOB must be between 15 and 100 years ago');
  7      END IF;
  8  END;
  9  /

Trigger created.

SQL>
SQL> CREATE OR REPLACE TRIGGER TRG_TEAM_GOALS
  2  AFTER INSERT OR UPDATE ON FOOTBALLER
  3  FOR EACH ROW
  4  BEGIN
  5      UPDATE TEAM
  6      SET GOALS_SCORED =
  7          CASE
  8              WHEN NVL(goals_scored,0) + NVL(:NEW.goals,0) - NVL(:OLD.goals,0) > goals_scored
  9              THEN NVL(goals_scored,0) + NVL(:NEW.goals,0) - NVL(:OLD.goals,0)
 10              ELSE GOALS_SCORED
 11          END
 12      WHERE ID = :NEW.TEAM_ID;
 13  END;
 14  /

Trigger created.
```

Figure 8. Constructing triggers. We check the temporal attributes and we ensure valid value of goals_scored of the team table.

```
SQL>
SQL> CREATE PROCEDURE PRC_TRG_UPP(
  2      TAB_NAME VARCHAR2,
  3      COLS_NAME SYS.ODCIVARCHAR2LIST
  4  ) AS
  5      COMMAND_TRG VARCHAR2(2000);
  6      BEGIN_STRUCTURE VARCHAR2(1000);
  7  BEGIN
  8      FOR i IN 1..COLS_NAME.COUNT LOOP
  9          BEGIN_STRUCTURE := BEGIN_STRUCTURE||':NEW.'||COLS_NAME(i)||':=UPPER(:NEW.'||COLS_NAME(i)||');'||CHR(10);
 10      END LOOP;
 11      COMMAND_TRG := 'CREATE OR REPLACE TRIGGER TRG_'||UPPER(TAB_NAME)||'_UPP'||
 12                     ' BEFORE INSERT OR UPDATE ON '||TAB_NAME||
 13                     ' FOR EACH ROW BEGIN '||BEGIN_STRUCTURE||' END;';
 14      EXECUTE IMMEDIATE COMMAND_TRG;
 15  END;
 16  /

Procedure created.

SQL>
SQL> BEGIN
  2      PRC_TRG_UPP('STADIUM', SYS.ODCIVARCHAR2LIST('CITY','NAME'));
  3      PRC_TRG_UPP('TEAM', SYS.ODCIVARCHAR2LIST('NAME','CITY_FOUNDED'));
  4      PRC_TRG_UPP('FOOTBALLER', SYS.ODCIVARCHAR2LIST('FIRST_NAME','SURNAME','COUNTRY'));
  5      PRC_TRG_UPP('SPONSOR', SYS.ODCIVARCHAR2LIST('NAME','TYPE','CITY_HQ'));
  6  END;
  7  /

PL/SQL procedure successfully completed.
```

Figure 9. Creation and execution of a procedure. The procedure help us build triggers that check and update the case of the values of certain attributes automatically.

```
SQL> SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE 'PRC%';

OBJECT_NAME
--------------------------------------------------------------------------------------------------------------------------------
PRC_TRG_UPP
```

Figure 10. Checking the created procedure which we use to create the upper case triggers.

```
TRIGGER_NAME
------------------------------------------------------------------------------------------------------------------------------------
TRG_FOOTBALLER_DOB
TRG_FOOTBALLER_UPP
TRG_SPONSORSHIP_DEPOSITDATE
TRG_SPONSOR_UPP
TRG_STADIUM_UPP
TRG_STADIUM_YEARBUILT
TRG_TEAM_GOALS
TRG_TEAM_UPP
TRG_TEAM_YEARFOUNDED

9 rows selected.
```

Figure 11.checking the creation of triggers. In total we have 9 triggers, along with the triggers we created by the procedure.

```
SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Northgate Arena', 'Manchester', 54000, 1991, 3);

1 row created.

SQL>
SQL> INSERT INTO STADIUM (CAPACITY, NAME, YEAR_BUILT, CITY)
  2  VALUES (18000, 'Riverside Park', 1978, 'Bristol');

1 row created.

SQL>
SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Estadio Sol', 'Seville', 62000, 1986, 5);

1 row created.

SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Harbour Field', 'Lisbon', 42000, 1994, 2);

1 row created.

SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Olympic Crescent', 'Athens', 70000, 1982, 4);

1 row created.

SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Nordstadt Stadion', 'Berlin', 48000, 1989, 3);

1 row created.

SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Marina Coliseum', 'Rio de Janeiro', 78000, 1985, 4);

1 row created.

SQL> INSERT INTO STADIUM VALUES (DEFAULT, 'Kyoto Garden', 'Kyoto', 32000, 1995, 5);

1 row created.
```

Figure 12. Population of table STADIUM.

```
SQL>
SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Grand Union Manchester', 'Manchester', 1902, 55, 25, 200);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Saint George Athletic', 'Oxford', 1933, 30, 45, 201);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'River Tigers', 'Bristol', 1950, 35, 25, 201);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Mega Athletic Bristol', 'Bristol', 1960, 18, 28, 201);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Panathenian Rovers', 'Athens', 1965, 60, 35, 204);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Thessaloniki United', 'Thessaloniki', 1970, 50, 20, 204);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Northern Olympians', 'Seville', NULL, 20, 27, 202);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Saint Bridget Rovers', 'Oxford', 1975, 34, 31, 202);

1 row created.

SQL> INSERT INTO TEAM VALUES (DEFAULT, 'Berlin Guardians', 'Berlin', 1940, 27, 30, 205);

1 row created.
```

Figure 13. Population of TEAM table with 14 rows. In the figure, only the top of the code is shown for space saving.

```
SQL> INSERT INTO FOOTBALLER (SURNAME, EMAIL, GAMES_PLAYED, COUNTRY, TEAM_ID, FIRST_NAME, GOALS)
  2  VALUES('Carter','liam.carter@lborough.com', 320, 'ENGLAND', 100, 'Liam', 5);

1 row created.

SQL>
SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Jon', 'Jackson', 'GK', 400, 0, 150000, 'jon.jackson@lborough.com', 'ENGLAND', '30-SEP-1985', 100);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Marcus', 'Johnson', 'FOR', 150, 20, 300000, 'marcus.johnson@lborough.com', 'USA', '01-JAN-1991', 100);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Owen', 'Peterson', 'MID', 190, 1, NULL, 'owen.peterson@lborough.com', 'ENGLAND', '21-JUN-1993', 100);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Mateo', 'Silva', 'MID', 210, 18, 250000, 'mateo.silva@grandunion.com', 'PORTUGAL', '05-JUL-1992', 101);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Pedro', 'Alvarez', 'FOR', NULL, 28, 410000, 'pedro.alvarez@grandunion.com', 'SPAIN', '11-JAN-1990', 101);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Ricardo', 'Mendes', DEFAULT, 142, 4, 95000, 'ricardo.mendes@grandunion.com', 'PORTUGAL', '13-SEP-1994', 101);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Alex', 'Rivers', 'FOR', 220, 31, 470000, 'alex.rivers@saintgeorge.com', 'ENGLAND', NULL, 102);

1 row created.

SQL> INSERT INTO FOOTBALLER VALUES (DEFAULT, 'Ethan', 'Jameson', 'MID', 140, 8, 115000, 'ethan.jameson@saintgeorge.com', 'ENGLAND', '30-APR-1994', 102);

1 row created.
```

Figure 14. Population of FOOTBALLER table with 40 rows. Note that only the top of the code executed is shown for space saving.

```
SQL> INSERT INTO SPONSOR (TYPE, NAME, CITY_HQ)
  2  VALUES ('ENERGY', 'Apex Energy', 'London');

1 row created.

SQL>
SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'BlueWave Electronics', 'ELECTRONICS', 'Seville');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Crest Financial', 'FINANCE', 'Zurich');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Delta Foods', 'FOOD', 'Lisbon');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Evergreen Logistics', 'LOGISTICS', 'Berlin');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Fjord Breweries', 'BEVERAGE', 'Oslo');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Global Tech', 'ELECTRONICS', 'Tokyo');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Harbor Insurance', 'INSURANCE', 'Athens');

1 row created.

SQL> INSERT INTO SPONSOR VALUES (DEFAULT, 'Zenith Motors', 'AUTOMOTIVE', 'Barcelona');

1 row created.
```

Figure 15. Population of SPONSOR table with 9 rows.

```
SQL> INSERT INTO SPONSORSHIP (SPONSOR_ID, FEE, DEPOSIT_DATE, TEAM_ID)
  2  VALUES (400, 50000, '01-JAN-2022', 100);

1 row created.

SQL>
SQL> INSERT INTO SPONSORSHIP VALUES (101, 401, '20-FEB-2022', 80000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (102, 403, '05-MAY-2022', 30000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (103, 401, '12-APR-2022', 90000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (104, 403, '03-MAR-2022', NULL);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (105, 407, '11-NOV-2021', 45000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (106, 402, '30-SEP-2022', 100000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (107, 405, '01-JUN-2022', 80000);

1 row created.

SQL> INSERT INTO SPONSORSHIP VALUES (108, 404, '18-MAY-2022', 35000);

1 row created.
```

Figure 16. Population of SPONSORSHIP table with 20 rows. Note that only the top part of the code is shown for space saving.

```
SQL> SELECT COUNT(*) FROM STADIUM;

  COUNT(*)
----------
         8

SQL> SELECT COUNT(*) FROM TEAM;

  COUNT(*)
----------
        14

SQL> SELECT COUNT(*) FROM FOOTBALLER;

  COUNT(*)
----------
        40

SQL> SELECT COUNT(*) FROM SPONSOR;

  COUNT(*)
----------
         9

SQL> SELECT COUNT(*) FROM SPONSORSHIP;

  COUNT(*)
----------
        20
```

Figure 17. Checking the populations of our tables.

```
SQL> CREATE OR REPLACE PROCEDURE PRC_COLS_FORMAT(TAB_NAME VARCHAR2) AS
  2      LINES_MAX_COLS_LEN VARCHAR2(200);
  3      MAX_COLS_LEN NUMBER;
  4      FORMATTING VARCHAR2(20);
  5  BEGIN
  6      DBMS_OUTPUT.PUT_LINE('-- FORMATTING COLUMNS OF ' || TAB_NAME || ':');
  7      FOR COLS IN (SELECT COLUMN_NAME AS NAME, DATA_TYPE AS TYPE FROM USER_TAB_COLUMNS WHERE TABLE_NAME = UPPER(TAB_NAME) ORDER BY COLUMN_ID) LOOP
  8          LINES_MAX_COLS_LEN := 'SELECT MAX(LENGTH(' || cols.name || ')) FROM ' || TAB_NAME;
  9          EXECUTE IMMEDIATE LINES_MAX_COLS_LEN INTO MAX_COLS_LEN;
 10     FORMATTING:=CASE
 11         WHEN COLS.TYPE='NUMBER' THEN LPAD('9', MAX_COLS_LEN, '9')
 12         ELSE 'A'||TO_CHAR(MAX_COLS_LEN)
 13     END;
 14          DBMS_OUTPUT.PUT_LINE('COLUMN '||COLS.NAME||' FORMAT '||FORMATTING);
 15      END LOOP;
 16  END;
 17  /

Procedure created.

SQL> CREATE OR REPLACE PROCEDURE PRC_START AS
  2  BEGIN
  3      DBMS_OUTPUT.PUT_LINE('
  4  ALTER SESSION SET NLS_DATE_FORMAT = ''DD-MON-YYYY'';
  5
  6  -- FORMATTING SESSION
  7  SET LINESIZE 200
  8  SET PAGESIZE 100
  9  ');
 10      FOR TABS IN (SELECT TABLE_NAME AS NAME FROM USER_TABLES ORDER BY TABLE_NAME) LOOP
 11     PRC_COLS_FORMAT(TABS.NAME);
 12     DBMS_OUTPUT.PUT_LINE('
 13  ');
 14      END LOOP;
 15      DBMS_OUTPUT.PUT_LINE('CLEAR SCREEN');
 16  END;
 17  /

Procedure created.
```

Figure 18. Creating two procedures to help us get the lines that we need to run when the session starts. The first procedure gets us the lines specifically for the formatting of the columns and the second wraps the first as well as for general formatting. In that way, when we EXEC PRC_START; we get the lines for formatting, we copy, paste and run them.

```
SQL> EXEC PRC_START;

ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MON-YYYY';

-- FORMATTING SESSION
SET LINESIZE 200
SET PAGESIZE 100

-- FORMATTING COLUMNS OF FOOTBALLER:
COLUMN ID FORMAT 999
COLUMN FIRST_NAME FORMAT A7
COLUMN SURNAME FORMAT A9
COLUMN POSITION FORMAT A3
COLUMN GAMES_PLAYED FORMAT 999
COLUMN GOALS FORMAT 99
COLUMN CONTRACT FORMAT 999999
COLUMN EMAIL FORMAT A33
COLUMN COUNTRY FORMAT A9
COLUMN DOB FORMAT A11
COLUMN TEAM_ID FORMAT 999


-- FORMATTING COLUMNS OF SPONSOR:
COLUMN ID FORMAT 999
COLUMN NAME FORMAT A20
COLUMN TYPE FORMAT A11
COLUMN CITY_HQ FORMAT A9


-- FORMATTING COLUMNS OF SPONSORSHIP:
COLUMN TEAM_ID FORMAT 999
COLUMN SPONSOR_ID FORMAT 999
COLUMN DEPOSIT_DATE FORMAT A11
COLUMN FEE FORMAT 999999


-- FORMATTING COLUMNS OF STADIUM:
COLUMN ID FORMAT 999
COLUMN NAME FORMAT A17
COLUMN CITY FORMAT A14
COLUMN CAPACITY FORMAT 99999
COLUMN YEAR_BUILT FORMAT 9999
COLUMN REVIEW_SCORE FORMAT 9


-- FORMATTING COLUMNS OF TEAM:
COLUMN ID FORMAT 999
COLUMN NAME FORMAT A22
COLUMN CITY_FOUNDED FORMAT A12
COLUMN YEAR_FOUNDED FORMAT 9999
COLUMN GOALS_SCORED FORMAT 999
COLUMN GOALS_CONCEDED FORMAT 99
COLUMN STADIUM_ID FORMAT 999


CLEAR SCREEN

PL/SQL procedure successfully completed.
```

Figure 19. Execution of the formatting procedures at the start of a session. We get the lines so we can copy, paste and run them to format our session.

```
SQL> SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE 'PRC%';

OBJECT_NAME
--------------------------------------------------------------------------------------------------------------------------------
PRC_COLS_FORMAT
PRC_START
PRC_TRG_UPP
```

Figure 20. Checking the creation of our procedures.

```
SQL> -- A. ALL THE RECORDS OF ONE TABLE
SQL> -- ----------------------------------------
SQL> SELECT * FROM TEAM;

  ID NAME                   CITY_FOUNDED YEAR_FOUNDED GOALS_SCORED GOALS_CONCEDED STADIUM_ID
---- --------------------- ------------ ------------ ------------ -------------- ----------
 100 LONDON BOROUGH FC      LONDON               1978           66                       200
 101 GRAND UNION MANCHESTER MANCHESTER           1902          105             25        200
 102 SAINT GEORGE ATHLETIC  OXFORD               1933          102             45        201
 103 RIVER TIGERS           BRISTOL              1950          127             25        201
 104 MEGA ATHLETIC BRISTOL  BRISTOL              1960           35             28        201
 105 PANATHENIAN ROVERS     ATHENS               1965           91             35        204
 106 THESSALONIKI UNITED    THESSALONIKI         1970           88             20        204
 107 NORTHERN OLYMPIANS     SEVILLE                             35             27        202
 108 SAINT BRIDGET ROVERS   OXFORD               1975           42             31        202
 109 BERLIN GUARDIANS       BERLIN               1940           32             30        205
 110 BARCA UNITED           BARCELONA            1958          133                       205
 111 EL MADRIDOS            MADRID               1935          158             29        206
 112 UNIDAD ESPANA          SEVILLE              1968          167             32        206
 113 KYOTO MARINERS         KYOTO                1998           67             18        207

14 rows selected.
```

Figure 21. Query A.

```
SQL> -- B. A PROJECTION WITH 3 COLUMNS IN REVERSE ALPHABETICAL ORDER
SQL> -- ------------------------------------
SQL> SELECT ID, NAME, CITY_FOUNDED, GOALS_SCORED
  2  FROM TEAM
  3  ORDER BY CITY_FOUNDED DESC, NAME DESC;

  ID NAME                   CITY_FOUNDED GOALS_SCORED
---- ---------------------- ------------ ------------
 106 THESSALONIKI UNITED    THESSALONIKI           88
 112 UNIDAD ESPANA          SEVILLE               167
 107 NORTHERN OLYMPIANS     SEVILLE                35
 102 SAINT GEORGE ATHLETIC  OXFORD                102
 108 SAINT BRIDGET ROVERS   OXFORD                 42
 101 GRAND UNION MANCHESTER MANCHESTER            105
 111 EL MADRIDOS            MADRID                158
 100 LONDON BOROUGH FC      LONDON                 66
 113 KYOTO MARINERS         KYOTO                  67
 103 RIVER TIGERS           BRISTOL               127
 104 MEGA ATHLETIC BRISTOL  BRISTOL                35
 109 BERLIN GUARDIANS       BERLIN                 32
 110 BARCA UNITED           BARCELONA             133
 105 PANATHENIAN ROVERS     ATHENS                 91

14 rows selected.
```

Figure 22. Query B.

```
SQL> -- C. RESTRICTION QUERIES WITH MULTIPLE CLAUSES
SQL> -- ----------------------------------------
SQL> -- 1. PLAYERS FROM A SPECIFIC COUNTRY
SQL> SELECT FIRST_NAME, SURNAME, COUNTRY
  2  FROM FOOTBALLER
  3  WHERE COUNTRY='GREECE';

FIRST_N SURNAME   COUNTRY
------- --------- ---------
GIORGOS TSATALAS  GREECE
ILIAS   TAGGIRIS  GREECE
PETROS  KARRAS    GREECE
```

Figure 23. Query C1.

```
SQL> -- 2. BEGINS WITH J, ENDS WITH SON
SQL> SELECT FIRST_NAME, SURNAME
  2  FROM FOOTBALLER
  3  WHERE SURNAME LIKE 'J%SON';

FIRST_N SURNAME
------- ---------
JON     JACKSON
MARCUS  JOHNSON
ETHAN   JAMESON
```

Figure 24. Query C2.

```
SQL> -- 3. ARE NOT FROM ENGLAND
SQL> SELECT FIRST_NAME, SURNAME, COUNTRY
  2  FROM FOOTBALLER
  3  WHERE COUNTRY<>'ENGLAND';

FIRST_N SURNAME   COUNTRY
------- --------- ---------
MARCUS  JOHNSON   USA
MATEO   SILVA     PORTUGAL
PEDRO   ALVAREZ   SPAIN
RICARDO MENDES    PORTUGAL
JAVIER  ANDERSON  ARGENTINA
DIEGO   SANTOS    BRAZIL
JAMAL   BROWN     USA
LUCAS   MOREIRA   BRAZIL
CARLOS  MENDES    PORTUGAL
MIGUEL  RAMOS     SPAIN
RAFAEL  DOSSANTOS BRAZIL
GIORGOS TSATALAS  GREECE
ILIAS   TAGGIRIS  GREECE
THIAGO  PINTO     BRAZIL
IVAN    KLEINS    CROATIA
SAMUEL  OKOYE     NIGERIA
MARCO   ROSSI     ITALY
PETROS  KARRAS    GREECE
HUGO    MENDEZ    URUGUAY
FELIPE  GOMES     BRAZIL
DANIEL  KOVAC     CROATIA
MATEUS  SILVEIRA  BRAZIL
HIRO    SATO      JAPAN
ELIAS   VAZQUEZ   MEXICO
PEDRO   MARTINEZ  SPAIN
DIEGO   ALVAREZ   ARGENTINA
JORGE   LOPEZ     ARGENTINA
SERGIO  CORTES    SPAIN
JAVIER  RODRIGUEZ ARGENTINA
ANA     MARQUEZ   SPAIN
YUKI    TANAKA    JAPAN
TAKUMI  SAKAMOTO  JAPAN
OMAR    KHAN      PAKISTAN
LUCAS   FERNANDES PORTUGAL

34 rows selected.
```

Figure 25. Query C3.

```
SQL> -- 4. BORN FROM APRIL TO SEPTEMBER
SQL> SELECT FIRST_NAME, SURNAME, DOB, TO_CHAR(DOB, 'MONTH')
  2  FROM FOOTBALLER
  3  WHERE TO_NUMBER(TO_CHAR(DOB, 'MM')) BETWEEN 4 AND 9;

FIRST_N SURNAME   DOB         TO_CHAR(DOB,'MONTH')
------- --------- ----------- ------------------------------------
JON     JACKSON   30-SEP-1985 SEPTEMBER
OWEN    PETERSON  21-JUN-1993 JUNE
MATEO   SILVA     05-JUL-1992 JULY
RICARDO MENDES    13-SEP-1994 SEPTEMBER
ETHAN   JAMESON   30-APR-1994 APRIL
JAVIER  ANDERSON  02-JUL-1989 JULY
DIEGO   SANTOS    03-JUL-1988 JULY
JAMAL   BROWN     05-MAY-1993 MAY
LUCAS   MOREIRA   10-JUL-1990 JULY
CARLOS  MENDES    14-MAY-1992 MAY
MIGUEL  RAMOS     04-JUN-1990 JUNE
RAFAEL  DOSSANTOS 24-AUG-1992 AUGUST
GIORGOS TSATALAS  02-MAY-1990 MAY
ILIAS   TAGGIRIS  02-AUG-1992 AUGUST
THIAGO  PINTO     09-JUL-1996 JULY
SAMUEL  OKOYE     23-JUL-1991 JULY
MARCO   ROSSI     07-JUL-1993 JULY
PETROS  KARRAS    02-AUG-1992 AUGUST
HUGO    MENDEZ    12-APR-1992 APRIL
DANIEL  KOVAC     21-JUN-1993 JUNE
MATEUS  SILVEIRA  18-JUL-1994 JULY
HIRO    SATO      04-MAY-1991 MAY
DIEGO   ALVAREZ   03-JUL-1988 JULY
JORGE   LOPEZ     19-MAY-1993 MAY
JAVIER  RODRIGUEZ 02-JUL-1989 JULY
ANA     MARQUEZ   08-SEP-1995 SEPTEMBER
YUKI    TANAKA    15-JUL-1995 JULY
LUCAS   FERNANDES 19-MAY-1995 MAY

28 rows selected.
```

Figure 26. Query C4.

```
SQL> -- 5. BRAZIL/ENGLAND, GK/MID, SURNAME STARTS WITH L BUT NOT CONTAINING 'i'
SQL> SELECT SURNAME, COUNTRY, POSITION
  2  FROM FOOTBALLER
  3  WHERE COUNTRY IN ('BRAZIL', 'ENGLAND') AND POSITION IN ('GK', 'MID') AND SURNAME LIKE 'L%' AND SURNAME NOT LIKE '%i%'
  4  ORDER BY COUNTRY, POSITION;

no rows selected
```

Figure 27. Query C5.

```
SQL> -- D. JOIN QUERIES
SQL> -- ---------------------------------------
SQL> -- 1. TWO TABLES
SQL> SELECT T.NAME, T.GOALS_SCORED, F.SURNAME, F.GOALS
  2  FROM TEAM T, FOOTBALLER F
  3  WHERE T.ID=F.TEAM_ID AND F.COUNTRY IN ('ENGLAND','BRAZIL');

NAME                   GOALS_SCORED SURNAME   GOALS
---------------------- ------------ --------- -----
LONDON BOROUGH FC                 66 CARTER        5
LONDON BOROUGH FC                 66 JACKSON       0
LONDON BOROUGH FC                 66 PETERSON      1
SAINT GEORGE ATHLETIC            102 RIVERS       31
SAINT GEORGE ATHLETIC            102 JAMESON       8
RIVER TIGERS                     127 SANTOS       40
RIVER TIGERS                     127 MOREIRA      29
MEGA ATHLETIC BRISTOL             35 DOSSANTOS     6
PANATHENIAN ROVERS                91 PINTO         3
THESSALONIKI UNITED               88 WATSON        0
BERLIN GUARDIANS                  32 GOMES         0
BARCA UNITED                     133 SILVEIRA     10

12 rows selected.
```

Figure 28. Query D1.

```
SQL> -- 2. THREE TABLES
SQL> SELECT S.NAME, S.CITY, T.NAME, T.CITY_FOUNDED, F.SURNAME, F.COUNTRY
  2  FROM FOOTBALLER F, TEAM T, STADIUM S
  3  WHERE T.ID=F.TEAM_ID AND S.ID=T.STADIUM_ID AND F.CONTRACT>70000 AND COUNTRY NOT IN ('BRAZIL', 'GREECE', 'ENGLAND');

NAME                 CITY           NAME                 CITY_FOUNDED SURNAME   COUNTRY
-------------------- -------------- -------------------- ------------ --------- ---------
NORTHGATE ARENA      MANCHESTER     LONDON BOROUGH FC    LONDON       JOHNSON   USA
NORTHGATE ARENA      MANCHESTER     GRAND UNION MANCHESTER MANCHESTER SILVA     PORTUGAL
NORTHGATE ARENA      MANCHESTER     GRAND UNION MANCHESTER MANCHESTER ALVAREZ   SPAIN
NORTHGATE ARENA      MANCHESTER     GRAND UNION MANCHESTER MANCHESTER MENDES    PORTUGAL
RIVERSIDE PARK       BRISTOL        SAINT GEORGE ATHLETIC OXFORD      ANDERSON  ARGENTINA
RIVERSIDE PARK       BRISTOL        RIVER TIGERS         BRISTOL      MENDES    PORTUGAL
RIVERSIDE PARK       BRISTOL        MEGA ATHLETIC BRISTOL BRISTOL     RAMOS     SPAIN
OLYMPIC CRESCENT     ATHENS         THESSALONIKI UNITED  THESSALONIKI KLEINS    CROATIA
OLYMPIC CRESCENT     ATHENS         THESSALONIKI UNITED  THESSALONIKI OKOYE     NIGERIA
ESTADIO SOL          SEVILLE        NORTHERN OLYMPIANS   SEVILLE      ROSSI     ITALY
ESTADIO SOL          SEVILLE        SAINT BRIDGET ROVERS OXFORD       MENDEZ    URUGUAY
NORDSTADT STADION    BERLIN         BERLIN GUARDIANS     BERLIN       KOVAC     CROATIA
NORDSTADT STADION    BERLIN         BARCA UNITED         BARCELONA    SATO      JAPAN
NORDSTADT STADION    BERLIN         BARCA UNITED         BARCELONA    VAZQUEZ   MEXICO
MARINA COLISEUM      RIO DE JANEIRO EL MADRIDOS          MADRID       MARTINEZ  SPAIN
MARINA COLISEUM      RIO DE JANEIRO EL MADRIDOS          MADRID       ALVAREZ   ARGENTINA
MARINA COLISEUM      RIO DE JANEIRO EL MADRIDOS          MADRID       LOPEZ     ARGENTINA
MARINA COLISEUM      RIO DE JANEIRO UNIDAD ESPANA        SEVILLE      CORTES    SPAIN
MARINA COLISEUM      RIO DE JANEIRO UNIDAD ESPANA        SEVILLE      RODRIGUEZ ARGENTINA
MARINA COLISEUM      RIO DE JANEIRO UNIDAD ESPANA        SEVILLE      MARQUEZ   SPAIN
KYOTO GARDEN         KYOTO          KYOTO MARINERS       KYOTO        TANAKA    JAPAN
KYOTO GARDEN         KYOTO          KYOTO MARINERS       KYOTO        SAKAMOTO  JAPAN
KYOTO GARDEN         KYOTO          KYOTO MARINERS       KYOTO        FERNANDES PORTUGAL

23 rows selected.

SQL>
SQL> SELECT TM.NAME, SP.NAME, SS.FEE
  2  FROM SPONSORSHIP SS
  3  JOIN TEAM TM ON SS.TEAM_ID = TM.ID
  4  JOIN SPONSOR SP ON SS.SPONSOR_ID = SP.ID
  5  ORDER BY SS.FEE DESC;

NAME                 NAME                      FEE
-------------------- -------------------- -------
MEGA ATHLETIC BRISTOL DELTA FOODS
BARCA UNITED         ZENITH MOTORS          150000
LONDON BOROUGH FC    BLUEWAVE ELECTRONICS   120000
BERLIN GUARDIANS     GLOBAL TECH            110000
THESSALONIKI UNITED  CREST FINANCIAL        100000
RIVER TIGERS         BLUEWAVE ELECTRONICS    90000
BARCA UNITED         BLUEWAVE ELECTRONICS    90000
GRAND UNION MANCHESTER BLUEWAVE ELECTRONICS  80000
NORTHERN OLYMPIANS   FJORD BREWERIES         80000
EL MADRIDOS          CREST FINANCIAL         70000
RIVER TIGERS         GLOBAL TECH             70000
SAINT BRIDGET ROVERS BLUEWAVE ELECTRONICS    60000
UNIDAD ESPANA        HARBOR INSURANCE        55000
LONDON BOROUGH FC    APEX ENERGY             50000
THESSALONIKI UNITED  BLUEWAVE ELECTRONICS    50000
KYOTO MARINERS       GLOBAL TECH             45000
PANATHENIAN ROVERS   HARBOR INSURANCE        45000
SAINT BRIDGET ROVERS EVERGREEN LOGISTICS     35000
KYOTO MARINERS       APEX ENERGY             30000
SAINT GEORGE ATHLETIC DELTA FOODS            30000

20 rows selected.
```

Figure 29. Query D2.

```
SQL> -- 3. MORE TABLES
SQL> SELECT SP.NAME, TM.NAME, ST.NAME, COUNT(FB.ID), SS.FEE
  2  FROM STADIUM ST, TEAM TM, FOOTBALLER FB, SPONSOR SP, SPONSORSHIP SS
  3  WHERE SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND TM.ID=FB.TEAM_ID AND ST.ID=TM.STADIUM_ID
  4  GROUP BY SP.NAME, TM.NAME, ST.NAME, SS.FEE;

NAME                 NAME                 NAME                 COUNT(FB.ID)     FEE
-------------------- -------------------- -------------------- ------------ -------
APEX ENERGY          LONDON BOROUGH FC    NORTHGATE ARENA                 4   50000
BLUEWAVE ELECTRONICS LONDON BOROUGH FC    NORTHGATE ARENA                 4  120000
BLUEWAVE ELECTRONICS GRAND UNION MANCHESTER NORTHGATE ARENA               3   80000
DELTA FOODS          SAINT GEORGE ATHLETIC RIVERSIDE PARK                 3   30000
BLUEWAVE ELECTRONICS RIVER TIGERS         RIVERSIDE PARK                 4   90000
GLOBAL TECH          RIVER TIGERS         RIVERSIDE PARK                 4   70000
DELTA FOODS          MEGA ATHLETIC BRISTOL RIVERSIDE PARK                2
HARBOR INSURANCE     PANATHENIAN ROVERS   OLYMPIC CRESCENT               3   45000
BLUEWAVE ELECTRONICS THESSALONIKI UNITED  OLYMPIC CRESCENT               3   50000
CREST FINANCIAL      THESSALONIKI UNITED  OLYMPIC CRESCENT               3  100000
FJORD BREWERIES      NORTHERN OLYMPIANS   ESTADIO SOL                    1   80000
BLUEWAVE ELECTRONICS SAINT BRIDGET ROVERS ESTADIO SOL                    2   60000
EVERGREEN LOGISTICS  SAINT BRIDGET ROVERS ESTADIO SOL                    2   35000
GLOBAL TECH          BERLIN GUARDIANS     NORDSTADT STADION              2  110000
BLUEWAVE ELECTRONICS BARCA UNITED         NORDSTADT STADION              3   90000
ZENITH MOTORS        BARCA UNITED         NORDSTADT STADION              3  150000
CREST FINANCIAL      EL MADRIDOS          MARINA COLISEUM                3   70000
HARBOR INSURANCE     UNIDAD ESPANA        MARINA COLISEUM                3   55000
APEX ENERGY          KYOTO MARINERS       KYOTO GARDEN                   4   30000
GLOBAL TECH          KYOTO MARINERS       KYOTO GARDEN                   4   45000

20 rows selected.
```

Figure 30. Query D3.

```
SQL> -- E. AGGREGATE FUNCTIONS
SQL> -- ------------------------------------
SQL> -- 1. FOOTBALLERS WITH SURNAME 'K...S'
SQL> SELECT FIRST_NAME, SURNAME, COUNTRY
  2  FROM FOOTBALLER
  3  WHERE SURNAME LIKE 'K%S';

FIRST_N SURNAME   COUNTRY
------- --------- ---------
IVAN    KLEINS    CROATIA
PETROS  KARRAS    GREECE
```

Figure 31. Query E1.

```
SQL> -- 2. TOTAL NUMBER OF PLAYERS PER TEAM
SQL> SELECT T.NAME, COUNT(*) PLAYERS
  2  FROM FOOTBALLER F, TEAM T
  3  WHERE T.ID=F.TEAM_ID
  4  GROUP BY T.NAME;

NAME                      PLAYERS
---------------------- ----------
LONDON BOROUGH FC               4
GRAND UNION MANCHESTER          3
SAINT GEORGE ATHLETIC           3
RIVER TIGERS                    4
MEGA ATHLETIC BRISTOL           2
PANATHENIAN ROVERS              3
THESSALONIKI UNITED             3
NORTHERN OLYMPIANS              1
SAINT BRIDGET ROVERS            2
BERLIN GUARDIANS                2
BARCA UNITED                    3
EL MADRIDOS                     3
UNIDAD ESPANA                   3
KYOTO MARINERS                  4

14 rows selected.

SQL>
SQL> SELECT T.ID, T.NAME, COUNT(F.SURNAME) NUMBER_OF_PLAYERS
  2  FROM FOOTBALLER F
  3  JOIN TEAM T ON F.TEAM_ID = T.ID
  4  GROUP BY T.NAME, T.ID;

  ID NAME                    NUMBER_OF_PLAYERS
---- --------------------- -----------------
 100 LONDON BOROUGH FC                      4
 101 GRAND UNION MANCHESTER                 3
 102 SAINT GEORGE ATHLETIC                  3
 103 RIVER TIGERS                           4
 104 MEGA ATHLETIC BRISTOL                  2
 105 PANATHENIAN ROVERS                     3
 106 THESSALONIKI UNITED                    3
 107 NORTHERN OLYMPIANS                     1
 108 SAINT BRIDGET ROVERS                   2
 109 BERLIN GUARDIANS                       2
 110 BARCA UNITED                           3
 111 EL MADRIDOS                            3
 112 UNIDAD ESPANA                          3
 113 KYOTO MARINERS                         4

14 rows selected.
```

Figure 32. Query E2.

```
SQL> -- 3. HIGHEST PAID FOOTBALLER
SQL> SELECT SURNAME, FIRST_NAME, CONTRACT
  2  FROM FOOTBALLER
  3  WHERE CONTRACT=(SELECT MAX(CONTRACT) FROM FOOTBALLER);

SURNAME   FIRST_N CONTRACT
--------- ------- --------
SANTOS    DIEGO     600000
ALVAREZ   DIEGO     600000
```

Figure 33. Query E3.

```
SQL> -- 4. HOW MANY STADIUMS WITH REVIEW SCORE LESS THAN 2
SQL> SELECT COUNT(*) COUNT_RS_LESS_2
  2  FROM STADIUM
  3  WHERE REVIEW_SCORE<2;

COUNT_RS_LESS_2
---------------
              1
```

Figure 34. Query E4.

```
SQL> -- 5. SURNAME, FIRST_NAME, ID AND THEIR TEAM IN ASCENDING ORDER
SQL> SELECT F.SURNAME, F.FIRST_NAME, F.ID, T.NAME
  2  FROM FOOTBALLER F, TEAM T
  3  WHERE T.ID=F.TEAM_ID
  4  ORDER BY SURNAME ASC, FIRST_NAME ASC;

SURNAME   FIRST_N   ID NAME
--------- ------- ---- ----------------------
ALVAREZ   DIEGO    931 EL MADRIDOS
ALVAREZ   PEDRO    905 GRAND UNION MANCHESTER
ANDERSON  JAVIER   909 SAINT GEORGE ATHLETIC
BROWN     JAMAL    911 RIVER TIGERS
CARTER    LIAM     900 LONDON BOROUGH FC
CORTES    SERGIO   933 UNIDAD ESPANA
DOSSANTOS RAFAEL   915 MEGA ATHLETIC BRISTOL
FERNANDES LUCAS    939 KYOTO MARINERS
GOMES     FELIPE   925 BERLIN GUARDIANS
JACKSON   JON      901 LONDON BOROUGH FC
JAMESON   ETHAN    908 SAINT GEORGE ATHLETIC
JOHNSON   MARCUS   902 LONDON BOROUGH FC
KARRAS    PETROS   923 SAINT BRIDGET ROVERS
KHAN      OMAR     938 KYOTO MARINERS
KLEINS    IVAN     919 THESSALONIKI UNITED
KOVAC     DANIEL   926 BERLIN GUARDIANS
LOPEZ     JORGE    932 EL MADRIDOS
MARQUEZ   ANA      935 UNIDAD ESPANA
MARTINEZ  PEDRO    930 EL MADRIDOS
MENDES    CARLOS   913 RIVER TIGERS
MENDES    RICARDO  906 GRAND UNION MANCHESTER
MENDEZ    HUGO     924 SAINT BRIDGET ROVERS
MOREIRA   LUCAS    912 RIVER TIGERS
OKOYE     SAMUEL   921 THESSALONIKI UNITED
PETERSON  OWEN     903 LONDON BOROUGH FC
PINTO     THIAGO   918 PANATHENIAN ROVERS
RAMOS     MIGUEL   914 MEGA ATHLETIC BRISTOL
RIVERS    ALEX     907 SAINT GEORGE ATHLETIC
RODRIGUEZ JAVIER   934 UNIDAD ESPANA
ROSSI     MARCO    922 NORTHERN OLYMPIANS
SAKAMOTO  TAKUMI   937 KYOTO MARINERS
SANTOS    DIEGO    910 RIVER TIGERS
SATO      HIRO     928 BARCA UNITED
SILVA     MATEO    904 GRAND UNION MANCHESTER
SILVEIRA  MATEUS   927 BARCA UNITED
TAGGIRIS  ILIAS    917 PANATHENIAN ROVERS
TANAKA    YUKI     936 KYOTO MARINERS
TSATALAS  GIORGOS  916 PANATHENIAN ROVERS
VAZQUEZ   ELIAS    929 BARCA UNITED
WATSON    DANIEL   920 THESSALONIKI UNITED

40 rows selected.
```

Figure 35. Query E5.

```
SQL> -- 6. FOOTBALLER (SURNAME, NAME, TEAM) HAS THE MOST GOALS
SQL> SELECT F.SURNAME, F.FIRST_NAME, T.NAME, F.GOALS
  2  FROM FOOTBALLER F, TEAM T
  3  WHERE T.ID=F.TEAM_ID AND F.GOALS=(SELECT MAX(GOALS) FROM FOOTBALLER);

SURNAME   FIRST_N NAME                   GOALS
--------- ------- ---------------------- -----
SANTOS    DIEGO   RIVER TIGERS              40
ALVAREZ   DIEGO   EL MADRIDOS               40
```

Figure 36. Query E6.

```
SQL> -- 7. TOTAL NUMBER OF TEAMS PER CITY
SQL> SELECT CITY_FOUNDED, COUNT(*) TEAMS_PER_CITY
  2  FROM TEAM
  3  GROUP BY CITY_FOUNDED
  4  ORDER BY COUNT(*) DESC, CITY_FOUNDED ASC;

CITY_FOUNDED TEAMS_PER_CITY
------------ --------------
BRISTOL                   2
OXFORD                    2
SEVILLE                   2
ATHENS                    1
BARCELONA                 1
BERLIN                    1
KYOTO                     1
LONDON                    1
MADRID                    1
MANCHESTER                1
THESSALONIKI              1

11 rows selected.
```

Figure 37. Query E7.

```
SQL> -- 8. TEAM WITH THE WORST GOAL DIFFERENCE
SQL> SELECT NAME, GOALS_SCORED-GOALS_CONCEDED DIFF
  2  FROM TEAM
  3  WHERE (GOALS_SCORED-GOALS_CONCEDED)=(SELECT MIN(GOALS_SCORED-GOALS_CONCEDED) FROM TEAM);

NAME                       DIFF
---------------------- ----------
BERLIN GUARDIANS               2
```

Figure 38. Query E8.

```
SQL> -- 9. NUMBER OF STADIUMS BUILT BETWEEN 1980 AND 1999
SQL> SELECT COUNT(*) COUNT_YEARBUILT_1980_1999
  2  FROM STADIUM
  3  WHERE YEAR_BUILT BETWEEN 1980 AND 1999;

COUNT_YEARBUILT_1980_1999
-------------------------
                        7

SQL>
SQL> SELECT COUNT(NAME) AS COUNT_OF_STADIUMS_1980_1999
  2  FROM STADIUM
  3  WHERE YEAR_BUILT >= 1980 AND YEAR_BUILT <= 1999;

COUNT_OF_STADIUMS_1980_1999
---------------------------
                          7
```

Figure 39. Query E9.

```
SQL> -- 10. PLAYERS THAT WERE BORN IN JULY
SQL> SELECT FIRST_NAME, SURNAME, COUNTRY, DOB
  2  FROM FOOTBALLER
  3  WHERE TO_CHAR(DOB, 'MON')='JUL'
  4  ORDER BY TO_CHAR(DOB,'DD') ASC;

FIRST_N SURNAME   COUNTRY   DOB
------- --------- --------- -----------
JAVIER  ANDERSON  ARGENTINA 02-JUL-1989
JAVIER  RODRIGUEZ ARGENTINA 02-JUL-1989
DIEGO   SANTOS    BRAZIL    03-JUL-1988
DIEGO   ALVAREZ   ARGENTINA 03-JUL-1988
MATEO   SILVA     PORTUGAL  05-JUL-1992
MARCO   ROSSI     ITALY     07-JUL-1993
THIAGO  PINTO     BRAZIL    09-JUL-1996
LUCAS   MOREIRA   BRAZIL    10-JUL-1990
YUKI    TANAKA    JAPAN     15-JUL-1995
MATEUS  SILVEIRA  BRAZIL    18-JUL-1994
SAMUEL  OKOYE     NIGERIA   23-JUL-1991

11 rows selected.

SQL>
SQL> SELECT FIRST_NAME, SURNAME, DOB
  2  FROM FOOTBALLER
  3  WHERE TO_NUMBER(TO_CHAR(DOB, 'MM')) = 7;

FIRST_N SURNAME   DOB
------- --------- -----------
MATEO   SILVA     05-JUL-1992
JAVIER  ANDERSON  02-JUL-1989
DIEGO   SANTOS    03-JUL-1988
LUCAS   MOREIRA   10-JUL-1990
THIAGO  PINTO     09-JUL-1996
SAMUEL  OKOYE     23-JUL-1991
MARCO   ROSSI     07-JUL-1993
MATEUS  SILVEIRA  18-JUL-1994
DIEGO   ALVAREZ   03-JUL-1988
JAVIER  RODRIGUEZ 02-JUL-1989
YUKI    TANAKA    15-JUL-1995

11 rows selected.
```

Figure 40. Query E10.

```
SQL> -- 11. NUMBER OF STADIUMS THAT DON'T HOST A TEAM
SQL> SELECT COUNT(*) NUMBER_OF_STADIUMS_NOT_HOSTING_TEAMS
  2  FROM STADIUM S
  3  WHERE S.ID NOT IN (SELECT DISTINCT T.STADIUM_ID FROM TEAM T);

NUMBER_OF_STADIUMS_NOT_HOSTING_TEAMS
------------------------------------
                                   1
```

Figure 41. Query E11.

```
SQL> -- 12. TEAM WITH NAME (%SAINT%)
SQL> SELECT NAME
  2  FROM TEAM
  3  WHERE NAME LIKE '%SAINT%';

NAME
----------------------
SAINT BRIDGET ROVERS
SAINT GEORGE ATHLETIC
```

Figure 42. Query E12.

```
SQL> -- 13. PLAYER FROM BRAZIL WHO HAS THE HIGHEST CONTRACT OR SCORED MOST GOALS
SQL> SELECT FIRST_NAME, SURNAME, CONTRACT, GOALS
  2  FROM FOOTBALLER
  3  WHERE COUNTRY='BRAZIL' AND
  4      (CONTRACT=(SELECT MAX(CONTRACT) FROM FOOTBALLER WHERE COUNTRY='BRAZIL') OR GOALS=(SELECT MAX(GOALS) FROM FOOTBALLER WHERE COUNTRY='BRAZIL'));

FIRST_N SURNAME   CONTRACT GOALS
------- --------- -------- -----
DIEGO   SANTOS      600000    40

SQL> -- VERIFICATION
SQL> SELECT FIRST_NAME, SURNAME, CONTRACT, GOALS
  2  FROM FOOTBALLER
  3  WHERE COUNTRY='BRAZIL';

FIRST_N SURNAME   CONTRACT GOALS
------- --------- -------- -----
DIEGO   SANTOS      600000    40
LUCAS   MOREIRA     430000    29
RAFAEL  DOSSANTOS    98000     6
THIAGO  PINTO        72000     3
FELIPE  GOMES       160000     0
MATEUS  SILVEIRA    210000    10

6 rows selected.
```

Figure 43. Query E13.

```
SQL> -- 14. PLAYERS WITH HIGHEST CONTRACT PER POSITION
SQL> SELECT F.SURNAME, F.FIRST_NAME, F.POSITION, F.CONTRACT
  2  FROM FOOTBALLER F, (SELECT POSITION POS, MAX(CONTRACT) MAXCON FROM FOOTBALLER GROUP BY POSITION) M
  3  WHERE F.POSITION=POS AND F.CONTRACT=MAXCON;

SURNAME   FIRST_N POS CONTRACT
--------- ------- --- --------
CORTES    SERGIO  DEF   110000
GOMES     FELIPE  GK    160000
SANTOS    DIEGO   FOR   600000
ALVAREZ   DIEGO   FOR   600000
SILVA     MATEO   MID   250000

SQL>
SQL> SELECT F.FIRST_NAME, F.SURNAME, F.POSITION, F.CONTRACT
  2  FROM FOOTBALLER F
  3  WHERE F.CONTRACT = (
  4      SELECT MAX(F2.CONTRACT)
  5      FROM FOOTBALLER F2
  6      WHERE F2.POSITION = F.POSITION)
  7  ORDER BY F.POSITION;

FIRST_N SURNAME   POS CONTRACT
------- --------- --- --------
SERGIO  CORTES    DEF   110000
DIEGO   ALVAREZ   FOR   600000
DIEGO   SANTOS    FOR   600000
FELIPE  GOMES     GK    160000
MATEO   SILVA     MID   250000
```

Figure 44. Query E14.

```
SQL> -- 15. USERNAME = <2 FIRST COUNTRY> + <4 FIRST FIRST_NAME> + <4 FIRST SURNAME>
SQL> SELECT ID, SUBSTR(COUNTRY,1,2)||SUBSTR(FIRST_NAME,1,4)||SUBSTR(SURNAME,1,4) USERNAME
  2  FROM FOOTBALLER;

  ID USERNAME
---- ----------------------------------------
 900 ENLIAMCART
 901 ENJONJACK
 902 USMARCJOHN
 903 ENOWENPETE
 904 POMATESILV
 905 SPPEDRALVA
 906 PORICAMEND
 907 ENALEXRIVE
 908 ENETHAJAME
 909 ARJAVIANDE
 910 BRDIEGSANT
 911 USJAMABROW
 912 BRLUCAMORE
 913 POCARLMEND
 914 SPMIGURAMO
 915 BRRAFADOSS
 916 GRGIORTSAT
 917 GRILIATAGG
 918 BRTHIAPINT
 919 CRIVANKLEI
 920 ENDANIWATS
 921 NISAMUOKOY
 922 ITMARCROSS
 923 GRPETRKARR
 924 URHUGOMEND
 925 BRFELIGOME
 926 CRDANIKOVA
 927 BRMATESILV
 928 JAHIROSATO
 929 MEELIAVAZQ
 930 SPPEDRMART
 931 ARDIEGALVA
 932 ARJORGLOPE
 933 SPSERGCORT
 934 ARJAVIRODR
 935 SPANAMARQ
 936 JAYUKITANA
 937 JATAKUSAKA
 938 PAOMARKHAN
 939 POLUCAFERN

40 rows selected.
```

Figure 45. Query E15.

```
SQL> -- 16. TOTAL OF FOOTBALLERS' CONTRACTS OF THE LEAGUE
SQL> SELECT SUM(CONTRACT) TOTAL_CONTRACT_AMOUNT FROM FOOTBALLER;

TOTAL_CONTRACT_AMOUNT
---------------------
              8536000
```

Figure 46. Query E16.

```
SQL> -- 17. SPONSOR SPENDING MOST MONEY
SQL> SELECT SP.NAME, SUM(SS.FEE)
  2  FROM SPONSOR SP, SPONSORSHIP SS
  3  WHERE SP.ID=SS.SPONSOR_ID
  4  GROUP BY SP.NAME
  5  HAVING SUM(SS.FEE)=(SELECT MAX(SUMFEE) FROM (SELECT SUM(SS.FEE) SUMFEE FROM SPONSORSHIP SS GROUP BY SS.SPONSOR_ID) M);

NAME                   SUM(SS.FEE)
---------------------- -----------
BLUEWAVE ELECTRONICS        490000
```

Figure 47. Query E17.

```
SQL> -- 18. TYPE OF BUSINESS THAT SPENDS MOST MONEY
SQL> SELECT SP.TYPE, SUM(SS.FEE)
  2  FROM SPONSOR SP, SPONSORSHIP SS
  3  WHERE SP.ID=SS.SPONSOR_ID
  4  GROUP BY SP.TYPE
  5  HAVING SUM(SS.FEE)=(SELECT MAX(XSUMFEE) FROM (
  6      SELECT SUM(SS.FEE) XSUMFEE FROM SPONSOR SP, SPONSORSHIP SS WHERE SP.ID=SS.SPONSOR_ID GROUP BY SP.TYPE) X);

TYPE        SUM(SS.FEE)
----------- -----------
ELECTRONICS      715000
```

Figure 48. Query E18.

```
SQL> -- 19. FOOTBALLERS WITH HIGHEST AVERAGE CONTRACTS PER SPONSOR
SQL> SELECT FB.FIRST_NAME, FB.SURNAME, TM.NAME, FB.CONTRACT, SP.NAME
  2  FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS
  3  WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID AND SP.ID IN (
  4      SELECT SP.ID FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS
  5      WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID
  6      GROUP BY SP.ID
  7      HAVING AVG(FB.CONTRACT)=(SELECT MAX(AVGCON) FROM (
  8     SELECT AVG(FB.CONTRACT) AVGCON FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS
  9     WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID
 10     GROUP BY SP.ID)));

FIRST_N SURNAME   NAME                   CONTRACT NAME
------- --------- ---------------------- -------- ----------------------
IVAN    KLEINS    THESSALONIKI UNITED      510000 CREST FINANCIAL
DANIEL  WATSON    THESSALONIKI UNITED       68000 CREST FINANCIAL
SAMUEL  OKOYE     THESSALONIKI UNITED       85000 CREST FINANCIAL
PEDRO   MARTINEZ  EL MADRIDOS              410000 CREST FINANCIAL
DIEGO   ALVAREZ   EL MADRIDOS              600000 CREST FINANCIAL
JORGE   LOPEZ     EL MADRIDOS               98000 CREST FINANCIAL

6 rows selected.

SQL> --VERIFICATION
SQL> SELECT SP.NAME, AVG(CONTRACT)
  2  FROM FOOTBALLER FB, TEAM TM, SPONSOR SP, SPONSORSHIP SS
  3  WHERE FB.TEAM_ID=TM.ID AND SP.ID=SS.SPONSOR_ID AND TM.ID=SS.TEAM_ID
  4  GROUP BY SP.NAME
  5  ORDER BY AVG(CONTRACT) DESC;

NAME                   AVG(CONTRACT)
---------------------- -------------
CREST FINANCIAL           295166.667
ZENITH MOTORS             273333.333
DELTA FOODS                   260600
BLUEWAVE ELECTRONICS        250187.5
FJORD BREWERIES               220000
GLOBAL TECH               208666.667
HARBOR INSURANCE          190666.667
APEX ENERGY               158833.333
EVERGREEN LOGISTICS            97500

9 rows selected.
```

Figure 49. Query E19.

```
SQL> -- 20. TEAM THAT GETS MOST FROM ELECTRONICS SPONSORS
SQL> SELECT TM.NAME, SUM(NVL(SS.FEE,0))
  2  FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS
  3  WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS'
  4  GROUP BY TM.NAME
  5  HAVING SUM(SS.FEE)=(SELECT MAX(XSUMFEE) FROM (SELECT SUM(NVL(SS.FEE,0)) XSUMFEE
  6      FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS
  7      WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS'
  8      GROUP BY TM.NAME) X);

NAME                   SUM(NVL(SS.FEE,0))
---------------------- ------------------
RIVER TIGERS                       160000

SQL> -- VERIFICATION
SQL> SELECT TM.NAME, SUM(NVL(SS.FEE,0))
  2  FROM TEAM TM, SPONSOR SP, SPONSORSHIP SS
  3  WHERE TM.ID=SS.TEAM_ID AND SP.ID=SS.SPONSOR_ID AND SP.TYPE='ELECTRONICS'
  4  GROUP BY TM.NAME
  5  ORDER BY SUM(NVL(SS.FEE,0)) DESC;

NAME                   SUM(NVL(SS.FEE,0))
---------------------- ------------------
RIVER TIGERS                       160000
LONDON BOROUGH FC                  120000
BERLIN GUARDIANS                   110000
BARCA UNITED                        90000
GRAND UNION MANCHESTER              80000
SAINT BRIDGET ROVERS                60000
THESSALONIKI UNITED                 50000
KYOTO MARINERS                      45000

8 rows selected.
```

Figure 50. Query E20.

```
SQL> DROP PROCEDURE PRC_START;

Procedure dropped.

SQL> DROP PROCEDURE PRC_COLS_FORMAT;

Procedure dropped.

SQL> DROP PROCEDURE PRC_TRG_UPP;

Procedure dropped.
```

Figure 51. Dropping all the procedures.

```
SQL> DROP TABLE SPONSORSHIP CASCADE CONSTRAINTS;

Table dropped.

SQL> DROP TABLE SPONSOR CASCADE CONSTRAINTS;

Table dropped.

SQL> DROP TABLE FOOTBALLER CASCADE CONSTRAINTS;

Table dropped.

SQL> DROP TABLE TEAM CASCADE CONSTRAINTS;

Table dropped.

SQL> DROP TABLE STADIUM CASCADE CONSTRAINTS;

Table dropped.
```

Figure 52. Dropping the tables with the constraints using CASCADE. With this way all the triggers are associated with the tables' attributes.

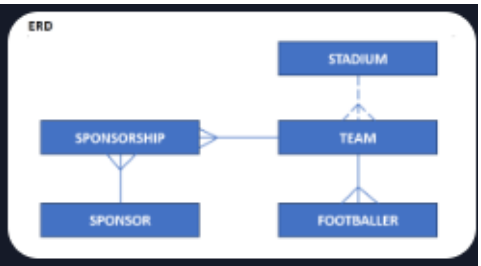
```
SQL> DROP SEQUENCE SEQ_STADIUM_ID;

Sequence dropped.

SQL> DROP SEQUENCE SEQ_TEAM_ID;

Sequence dropped.

SQL> DROP SEQUENCE SEQ_FOOTBALLER_ID;

Sequence dropped.

SQL> DROP SEQUENCE SEQ_SPONSOR_ID;

Sequence dropped.
```

Figure 53. Dropping the sequences.

```
SQL> SELECT OBJECT_NAME FROM USER_PROCEDURES WHERE OBJECT_NAME LIKE 'PRC%';

no rows selected

SQL>
SQL> SELECT TRIGGER_NAME FROM USER_TRIGGERS WHERE TRIGGER_NAME LIKE 'TRG%';

no rows selected

SQL>
SQL> SELECT CONSTRAINT_NAME FROM USER_CONSTRAINTS
  2  WHERE CONSTRAINT_NAME NOT LIKE 'BIN%' AND CONSTRAINT_NAME NOT LIKE 'SYS%';

no rows selected

SQL>
SQL> SELECT TNAME FROM TAB WHERE TNAME NOT LIKE 'BIN%';

no rows selected

SQL>
SQL> SELECT SEQUENCE_NAME FROM USER_SEQUENCES;

no rows selected
```

Figure 54. Checking that everything from the schema is dropped.

APPENDIX B



Should all the foreign keys be not null; And if so, why; In our schema we have this ERD. In this only the stadiums are allowed not to host a team. Please give a concise answer in a pargraph of about 150 words.

Not all foreign keys need to be `NOT NULL`; it depends on whether the relationship they represent is mandatory or optional. In your ERD, the only optional relationship is between STADIUM and TEAM — meaning a stadium may exist without hosting a team. Therefore, the foreign key in TEAM referencing STADIUM should allow `NULL` values. All other relationships, such as between SPONSORSHIP and TEAM or between FOOTBALLER and TEAM, are structurally mandatory, implying that each sponsorship must be linked to a team and each footballer must belong to one. In these cases, the foreign keys should be declared `NOT NULL` to enforce referential integrity and prevent orphan records. Making foreign keys `NOT NULL` ensures that the child entity cannot exist without a valid reference to its parent, which aligns with your schema's logic. Optional relationships should allow `NULL` to reflect real-world flexibility without violating database consistency.

But all the teams must be hosted in a specific stadium. So do i still leave the foreign key stadium_id of the team as able to be null; Plese give a concise answer of 80 words.

No — if every team must be hosted in a specific stadium, then the foreign key `stadium_id` in the TEAM entity should be declared `NOT NULL`. This enforces the mandatory relationship and ensures that no team record can exist without referencing a valid stadium. Allowing `NULL` would contradict your schema's logic by permitting teams without stadiums. Declaring it `NOT NULL` maintains referential integrity and aligns with the real-world rule that every team must have a designated stadium.

Figure 55. AI's help on clarifying whether we must declare foreign keys as not null.

I know this syntax:
ALTER TABLE <TAB> ADD CONSTRAINT <CONSTRAINT>;
Is there a compact way to add constraints in bulk in general or per table; Give me a very concise answer of under 100 words. Im using Oracle to build my schema. Give me an example with hypothetical tables and columns.

Yes — Oracle allows bulk constraint addition using **one** `ALTER TABLE … ADD (…)` block containing multiple comma-separated constraints. This is the only compact post-creation method.

Example:

Sql Copy

```sql
ALTER TABLE EMPLOYEE ADD (
  CONSTRAINT PK_EMP PRIMARY KEY (EMP_ID),
  CONSTRAINT UQ_EMP_EMAIL UNIQUE (EMAIL),
  CONSTRAINT CHK_EMP_SAL CHECK (SALARY > 0)
);

ALTER TABLE DEPARTMENT ADD (
  CONSTRAINT PK_DEPT PRIMARY KEY (DEPT_ID),
  CONSTRAINT UQ_DEPT_NAME UNIQUE (NAME)
);
```

Figure 56. AI's help on constructing constraints in bulk for saving space and time.

How can we ensure the upper case of the values of one attribute without worrying about the case of the vaues when we insert them in the table; Please give as much as a concise answer as you can; Give a hypothetical example, without stating any code and keep in mind that we work in oracle.

See my thinking

Use Oracle to transform values before storage so inserts can be any case. Common options: a **BEFORE INSERT/UPDATE trigger** that converts the incoming value to `UPPER(...)` so the stored value is always uppercase; or a **generated (virtual or stored) column** defined as the uppercase form and treated as the authoritative attribute. A CHECK constraint can only reject non-uppercase values, not convert them.

Figure 57. AI's help on ensuring that new values are inserted in the right case in our tables.

Give me the general structure of a trigger in a format like this:
CREATE TABLE <TAB>(
<COL1> <DATA_TYPE> <NOT NULL-DEFAULT>,
<COL2> <DATA_TYPE> <NOT NULL-DEFAULT>);
Also explain keypoints i should know about different basic keywords-clauses in a very brief manner. keep your answer very concise ang general. Mind that we work in oracle.

Here is the **general trigger structure** in the same style and level of abstraction as your table template:

Code Copy

```
CREATE [OR REPLACE] TRIGGER <TRIGGER_NAME>
    <BEFORE | AFTER> <INSERT | UPDATE | DELETE>
    ON <TABLE_NAME>
    [FOR EACH ROW]
    [WHEN (<CONDITION>)]
BEGIN
    <STATEMENT(S) USING :NEW, :OLD, := ASSIGNMENTS>;
END;
```

Keypoints (very brief)

- **BEFORE / AFTER** — defines when the trigger fires relative to the DML action.
- **INSERT / UPDATE / DELETE** — defines which operation activates it.
- **ON <TABLE>** — binds the trigger to a specific table.
- **FOR EACH ROW** — row-level trigger; without it, statement-level.
- **WHEN** — optional condition restricting execution.
- **:NEW / :OLD** — access to incoming and existing row values.
- **:=** — assignment operator used inside the trigger body.

Figure 58. AI's help to construct triggers.

ALTER TABLE STADIUM ADD CONSTRAINT CHK_STADIUM_YEARBUILT CHECK (TO_CHAR(YEAR_BUILT)<=TO_CHAR(SYSDATE, 'YYYY'));
We tried to build the constraint above but Oracle does not allow this. Please explain why this is not valid and how we can possibly overcome this (if not with a different constraint sructure maybe with a trigger). Your explanation must be as brief as possible, without giving us the code. Give us the xplanation and if need be, give us a general formatting structure.

Oracle rejects that constraint because a **CHECK constraint must be deterministic and based only on the row's own values.**
`SYSDATE` is **non-deterministic**, so Oracle cannot allow it inside a CHECK condition. That's why the constraint is invalid.

To enforce this rule, you must either:

- **Store only the row value** in the constraint (e.g., compare against a fixed limit), or
- **Use a BEFORE INSERT/UPDATE trigger** that checks the value against the current date and raises an error if it violates the rule.

General trigger structure:

Code Copy

```
CREATE TRIGGER <TRG_NAME>
    BEFORE INSERT OR UPDATE ON <TABLE>
    FOR EACH ROW
BEGIN
    <VALIDATION USING :NEW VALUES>;
END;
```

Figure 59. AI's help on why SYSDATE dependent constraints don't work and how to build a trigger instead that ensures valid temporal values.

ALTER TABLE FOOTABLLER ADD CONSTRAINT CHK_FOOTBALLER_POSITION CHECK (POSITION IN ('GK','DEF','MID','FOR'));
In the constraint above, we state that the column POSITION must contain one of those string literals. Do we need to ensure the upper case of the inserted values of POSITION, or is it redundant given that the list of strings in the contraint above has only upper case; Please be very concise and mind that we work on Oracle.

No extra uppercase enforcement is needed **only if you want the check to be case-sensitive.** Oracle compares strings exactly, so `'gk'`, `'Gk'`, or `'gK'` will **fail** the constraint because they don't match the uppercase literals.

If you want to accept any case but store or validate as uppercase, you must enforce uppercase separately (e.g., trigger). Otherwise, the constraint works as intended and rejects non-uppercase values.

Would you like a compact trigger template for auto-uppercasing or a case-insensitive constraint pattern?

Figure 60. AI's help on whether we need to check the case of an attribute that we already have a check constraint about, having certain values.

```
SQL> INSERT INTO FOOTBALLER (SURNAME, EMAIL, GAMES_PLAYED, COUNTRY, TEAM_ID, FIRST_NAME, GOALS)
  2  VALUES('Carter','liam.carter@lborough.com', 320, 'ENGLAND', 100, 'Liam', 5);
INSERT INTO FOOTBALLER (SURNAME, EMAIL, GAMES_PLAYED, COUNTRY, TEAM_ID, FIRST_NAME, GOALS)
            *
ERROR at line 1:
ORA-04091: table c##TSATALAS.FOOTBALLER is mutating, trigger/function may not
see it
ORA-06512: at "c##TSATALAS.TRG_TEAM_GOALS", line 4
ORA-04088: error during execution of trigger 'c##TSATALAS.TRG_TEAM_GOALS'
```

Figure 61. Issue with a trigger. We could not insert values into the table FOOTBALLEER.

```
ES_PLAYED, COUNTRY, TEAM_ID, FIRST_NAME,
 320, 'ENGLAND', 100, 'Liam', 5);
AYED, COUNTRY, TEAM_ID, FIRST_NAME, GOAL


ing, trigger/function may not

 4
#TSATALAS.TRG_TEAM_GOALS'
```

```
CREATE TRIGGER TRG_TEAM_GOALS
AFTER INSERT OR UPDATE ON FOOTBALLER
FOR EACH ROW
DECLARE
    GOALS_SUM NUMBER;
BEGIN
    SELECT NVL(SUM(F.GOALS),0)
    INTO GOALS_SUM
    FROM FOOTBALLER F
    WHERE F.TEAM_ID = :NEW.TEAM_ID;

    UPDATE TEAM T
    SET T.GOALS_SCORED = GOALS_SUM
    WHERE T.ID = :NEW.TEAM_ID AND GOALS_SUM > T.GOALS_SCORED;
END;
/
```

Why does oracle throw this error; How can we solve it. Please give us a concise answer. Explain to us what we did wrong without giving us any code. If need be, provide only a general structure format of code.

Oracle throws the **mutating table error** because your **row-level trigger queries the same table (FOOTBALLER)** that is currently being modified. During `AFTER INSERT OR UPDATE`, Oracle restricts access to the changing table to prevent inconsistent reads.

To solve this, avoid querying the mutating table inside the trigger. Instead, use a **delta-based update** that relies only on `:NEW` and `:OLD` values, without any SELECT from the same table.

General structure format:

Code Copy

```
CREATE OR REPLACE TRIGGER <TRIGGER_NAME>
AFTER INSERT OR UPDATE ON <TABLE>
FOR EACH ROW
BEGIN
    UPDATE <OTHER_TABLE>
    SET <TARGET_COLUMN> =
        CASE
            WHEN <DELTA_CONDITION>
            THEN <EXPRESSION USING :NEW and :OLD>
            ELSE <TARGET_COLUMN>
        END
    WHERE <MATCHING_CONDITION>;
END;
/
```

This avoids querying the mutating table and safely updates the related table using only the incoming row data.

Figure 62. AI's help on the issue with the trigger at Figure 61.

We want to construct several triggers that ensure the upper case insertion of specific columns. Thats why we want to construct a procedure. The procedure will have as arguments the table name and the column names and it will construct automatically a trigger for upper case for the columns specified. Please give us a general structure formatting without giving any specific code. Be as concise and simple as possible without giving redundant information. Also, give us the keypoints of the most basic and important keywords-clauses of a procedure.

Here is the **general structure** you need — abstract, minimal, and suitable for an assignment-style answer.

General structure of such a procedure

Code Copy

```
CREATE PROCEDURE <PROC_NAME>(
    P_TABLE_NAME  IN <STRING_TYPE>,
    P_COLUMN_LIST IN <LIST_TYPE>
)
IS
BEGIN
    <BUILD TRIGGER TEXT USING PARAMETERS>
    <EXECUTE DYNAMIC CREATION OF TRIGGER>
END;
/
```

What the procedure conceptually does

- **Receives** a table name and a list of column names.
- **Constructs** a trigger definition as text.
- The trigger enforces **uppercase assignment** for each specified column during insert/update.
- **Executes** the constructed trigger dynamically.

Keypoints of important procedure clauses

- **CREATE OR REPLACE PROCEDURE** — defines the stored procedure.
- **IN parameters** — input arguments (table name, column list).
- **IS / BEGIN / END** — marks the procedure body.
- **DECLARE section** — optional area for local variables.
- **:=** — assignment operator inside the procedure.
- **Dynamic execution** — used to create the trigger text at runtime.

If you want, I can also give you a **general trigger structure** that this procedure would generate.

Figure 63. AI's help to build a procedure that constructs triggers ensuring the upper case insertion of specific attributes' values on a specific table.

We know INSERT INTO <TAB> VALUES <...>;
Is there a way to write more compactly (in general or per table) the insertions in Oracle; Give us a concise answer.

No — there is **no more compact form** than
`INSERT INTO <TAB> VALUES (...);`

Figure 64. Checking if there is a more compact way to insert values into a table.

tell me how to format Oracle in order for my queries to give compact answers. If my query contains many columns, they either overlap or they are off screen. I want the columns to be able to fit inside my screen without overlapping. Please be very conscise and brief.

Here is the *compact SQL|Plus formatting pack*, exactly as you requested — concise, brief, and including column formats:

Code Copy

```
SET LINESIZE 150        -- sets total line width to fit your screen
SET PAGESIZE 50000      -- removes page breaks for continuous output
SET WRAP OFF            -- prevents lines from wrapping
SET WORD_WRAPPED OFF    -- prevents word-based wrapping
SET NEWPAGE 0           -- removes blank pages
SET SPACE 1             -- minimizes spacing between columns

COLUMN col FORMAT A<n>  -- sets fixed width for VARCHAR2/DATE columns
COLUMN col FORMAT 999999999  -- sets numeric width to prevent overlap
```

If you want, I can generate a **ready-to-use formatting script** you can drop into any assignment.

Figure 65. AI's help on formatting the session and the columns.

I have a procedure with some issue. Please without providing any code, pinpoint the issues and write me a brief explanation.

```
CREATE OR REPLACE PROCEDURE PRC_COLS_FORMAT(TAB_NAME
VARCHAR2) AS
   QUERY_MAX_COLS_LEN VARCHAR2(200);
   MAX_COLS_LEN NUMBER;
   FORMATTING VARCHAR2(20);
   COL_FORMAT VARCHAR2(40);
BEGIN
   FOR COLS IN (SELECT COLUMN_NAME AS NAME, DATA_TYPE AS
TYPE FROM USER_TAB_COLUMNS WHERE TABLE_NAME =
UPPER(TAB_NAME) ORDER BY COLUMN_ID) LOOP
      QUERY_MAX_COLS_LEN := 'SELECT MAX(LENGTH(' || COLS.NAME ||
')) FROM ' || TAB_NAME;
      EXECUTE IMMEDIATE QUERY_MAX_COLS_LEN INTO
MAX_COLS_LEN;
   FORMATTING:=CASE
      WHEN COLS.TYPE='NUMBER' THEN LPAD('9', MAX_COLS_LEN, '9')
      ELSE 'A'||TO_CHAR(MAX_COLS_LEN)
   END;
   COL_FORMAT:='COLUMN '||COLS.NAME||' FORMAT '||FORMATTING;
      EXECUTE IMMEDIATE COL_FORMAT;
   END LOOP;
END;
/
```

The core issue is that your procedure tries to execute *SQL*|Plus client commands* (`COLUMN … FORMAT`) inside PL/SQL. SQL*Plus formatting directives are not SQL and cannot be run through `EXECUTE IMMEDIATE`, so the procedure fails even though the rest of the logic is syntactically valid.

Figure 66. AI's help on the formatting procedure. We cannot execute with a procedure client commands.

Can you explain to us the syntax of the modern way to join tables; Please give us a concise and hypothetical example with 4 tables (tab4 depends on tab3 and tab2, tab3 depends on tab1). Please be very concise!
We already know how to join tables with the WHERE clause and we know how to join tables with: FROM <tab1> JOIN <tab2> ON <id equality>.
Please be very concise

Here's the **modern ANSI join syntax**, explained **very concisely**, with a **4-table dependency chain** exactly as you described.

⭐ Modern JOIN Syntax (very concise)

You join each table explicitly using:

Code ^ Copy

```
FROM t1
JOIN t2 ON t1.col = t2.col
JOIN t3 ON t2.col = t3.col
JOIN t4 ON (t3.col = t4.col AND t2.col = t4.other_col)
```

Each `JOIN` shows the dependency clearly and keeps conditions out of the `WHERE` clause.

- explicit join clauses
- clear ON conditions
- dependency-driven chaining
- no mixing with WHERE for joins

Figure 67. AI's help on understanding better join syntax with JOIN ON clause.

can you tell us whether the keyword LIMIT is available in oracle. If not, can you tell us if there is an alternative in oracle; Please be very concise with your answer.

Oracle **does not support** `LIMIT`.

The concise Oracle alternative is:

- `FETCH FIRST n ROWS ONLY` (modern ANSI syntax)
- `ROWNUM` (older Oracle-specific method)

Both achieve the same effect as `LIMIT`.

Figure 68. AI's help on whether LIMIT is available in Oracle.

I want some clarification on when to put a subquery in a HAVING or a WHERE clause. What are the differences and similarities; Please give a compact and consice answer.

Here's the **compact, concise** clarification:

- **WHERE** → filters **rows before grouping**; use when the subquery applies to **individual rows.**
- **HAVING** → filters **groups after grouping**; use when the subquery applies to **aggregated results.**
- **Similarity** → both restrict results, but they operate at **different stages** of query processing.

Figure 69. AI's help on when to put a subquery in a HAVING or a WHERE clause.

I want some clarification on when to put a subquery in a FROM or in WHERE clause. What are the differences and similarities. Please give a very compact and concise answer of max 100 words. Also give me a compact hypothetical example showcasing an instance when a subquery in WHERE clause isnt suitable and must be put in a FROM clause and vice versa.

Here's the **very compact ≤100-word explanation** plus **tight, meaningful examples**:

FROM-subquery → use when you need a **derived table** (pre-aggregated or reshaped data) that the outer query must **join to.**
WHERE-subquery → use when you only need a **boolean filter** on the outer rows.
Similarity → both restrict data; they operate at **different stages.**

When WHERE is not suitable → must use FROM

You need aggregated data **per group** and must join to it:

Code Copy

```
SELECT e.name, x.avg_sal
FROM emp e
JOIN (SELECT dept_id, AVG(salary) avg_sal
      FROM emp GROUP BY dept_id) x
  ON e.dept_id = x.dept_id;
```

When FROM is not suitable → must use WHERE

You only need a **filter**, not a table:

Code Copy

```
SELECT *
FROM emp
WHERE salary > (SELECT AVG(salary) FROM emp);
```

Figure 70. AI's help on when to put a subquery in a FROM clause and when to put it in a WHERE.

SELECT T. NAME, T. CITY_FOUNDED, AVG(F.CONTRACT) FROM TEAM T, FOOTBALLER F WHERE T.ID=F.TEAM_ID GROUP BY T.NAME ORDER BY AVG(F.CONTRACT);
There is an issue with the above query. Can you explain briefly and compactly the reason;

The issue is simple:

You **selected columns that are not in the GROUP BY.**

In Oracle (and standard SQL), every non-aggregated column in the SELECT list **must** appear in the GROUP BY clause.
Your query groups only by `T.NAME`, but also selects `T.CITY_FOUNDED`, which is neither grouped nor aggregated — causing an error or undefined results.

If you want it valid, you must either **add all non-aggregated columns to GROUP BY** or **remove them.**

Figure 71. AI's help on a GROUP BY query.

If we want an descending order in oracle we need to write ORDER BY col DESC. By default though, is it necessary to state ORDER BY <col> ASC; Or is it redundant; Please be very concise.

No — writing `ASC` **is redundant.**
In Oracle, `ORDER BY col` **already means ascending order,** so adding `ASC` changes nothing.

- DESC must be stated explicitly
- ASC is the default and optional

If you want, I can show a compact example comparing both styles.

Figure 72. AI's help on whether ASC is needed to be stated.